
Lehrstuhl für Informatik 3

Rechnerarchitektur

Aiswarya Mukherjee

Implementation of a RISC-V Compatible IEEE 754 Floating Point Unit in FPGAs

Masterthesis in Information and Communication Technology

1. April 2026

Please cite as:

Aiswarya Mukherjee, "Implementation of a RISC-V Compatible IEEE 754 Floating Point Unit in FPGAs," Master's Thesis (Masterarbeit), University of Erlangen, Dept. of Computer Science, April 2026.

Implementation of a RISC-V Compatible IEEE 754 Floating Point Unit in FPGAs

Masterthesis in Information and Communication Technology

vorgelegt von

Aiswarya Mukherjee

geb. am 06. January 1998
in Calcutta, India

angefertigt am

**Lehrstuhl für Informatik 3
Rechnerarchitektur**

**Department Informatik
Friedrich-Alexander-Universität Erlangen-Nürnberg**

Betreuer: **Mr. Farhad Ebrahimiandaryani**
Betreuender Hochschullehrer: **Prof. Dr.-Ing. Dietmar Fey**

Beginn der Arbeit: **1. October 2025**
Abgabe der Arbeit: **1. April 2026**

Erklärung

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde.

Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Declaration

I declare that the work is entirely my own and was produced with no assistance from third parties.

I certify that the work has not been submitted in the same or any similar form for assessment to any other examining body and all references, direct and indirect, are indicated as such and have been cited accordingly.

(Aiswarya Mukherjee)
Erlangen, 1. April 2026

Abstract

The incorporation of floating-point instructions into the RISC-V architecture greatly increases its ability to process computationally demanding workloads, which are essential in applications such as scientific computing, machine learning, and real-time data processing.

Implementation of pipelined floating-point units enables efficient execution of floating-point arithmetic operations by allowing multiple instructions to be processed concurrently, thereby improving overall processor performance while maintaining IEEE-754 compliant numerical accuracy. Out of the arithmetic units, division module is the most time consuming . Designing a proper floating point division module improves the overall performance and also improves the frequency.

A fully functional RISC-V compatible 32 bit single precision IEEE 754 FPU has been developed and implemented successfully on FPGA. The design includes all the instructions necessary to carry out floating point operations.

This research presents the design and implementation of a fully three-stage pipelined floating-point arithmetic unit. Furthermore, a comparison between different division architectures is performed to identify the most compatible and efficient implementation for FPGA realization.

The proposed architecture was developed based on the NOVACore in-order RISC-V processor and was successfully integrated into the existing pipeline. Initially, the processor supported the 32-bit RV32IZmmul-Zicsr instruction set architecture (ISA). The architecture was subsequently extended to support the RV32IMF-Zicsr ISA by incorporating floating-point functionality. The complete architecture was verified through simulation. A .coe file containing the program instructions was generated from C code using the RISC-V toolchain and loaded into the instruction memory for execution. The simulation results confirm that all instructions were executed correctly and that the integrated floating-point unit operates as expected. These results demonstrate the successful integration and functional verification of the extended processor architecture.

The final architecture was implemented on the Ultra96-U2 FPGA platform. The results confirmed the correct functionality of the proposed design.

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivation	1
1.3	Problem Statement	2
1.4	Development Environment	2
1.4.1	High-Level Modeling Environment	3
1.4.2	RTL Design and Synthesis Tools	3
1.4.3	Verification and Analysis Tools	3
1.4.4	Development Infrastructure	3
1.5	Development Stages	4
1.6	Literature Review	5
2	Architecture Overview	8
2.1	RISC-V Architecture	8
2.1.1	Pipelining	8
2.1.2	In Order Processor	10
2.2	RISC-V Floating-Point Extensions	11
2.2.1	Key Features of the F-Extension	11
2.3	IEEE 754 Standard	12
2.3.1	Overview	12
2.3.2	Standard Formats	13
2.3.3	Special Values	14
2.3.4	Rounding Modes	14
2.4	Field Programmable Gate Array	15
2.4.1	FPGA Architecture Overview	15
2.4.2	Key Characteristics and Advantages:	16
3	Analysis of the Initial Floating-Point Architecture	17
3.1	Initial System Architecture	17
3.2	FPGA-Based Processor Implementation	18

3.2.1	Zed board	19
3.2.2	Test Setup	21
3.3	Observations	22

I Design of a Pipelined IEEE 754 Compliant Floating-Point Arithmetic Unit for a RISC-V Processor 24

4	Floating-Point Arithmetic Unit Design	25
4.1	Overview of Arithmetic Units	25
4.1.1	Methodology	25
4.1.2	Special Case Handling	25
4.1.3	Rounding Modes	26
4.1.4	Guard, Round, and Sticky Bits	27
4.2	Three-Stage Pipelined Architecture	28
4.2.1	Execute Stage Design	28
4.2.2	Results	30
4.2.3	Division Architecture Alternatives	31
4.2.3.1	Various Division Algorithms	31
5	Floating-Point Division Architectures	33
5.1	Newton-Raphson Divider	33
5.1.1	Algorithm and Architecture	33
5.1.2	Verification Results	37
5.2	Iterative Restoring Floating-Point Divider	39
5.2.1	Algorithm and Control Design	39
5.2.2	Pipeline Stages of the Iterative Division Unit	39
5.2.3	Verification Results	42
5.3	Final Selection of the Division Architecture	43
5.3.1	Simulation Results of the Entire Design Using Restoring Iterative Division	43
5.3.2	Simulation Results of The Entire Design Using NR Division	44
5.3.3	Memory Utilization	45
5.3.4	Comparison	45
6	Final Design Implementation on FPGA	48
6.1	Intellectual Property	48
6.1.1	Clocking Wizard	48
6.1.2	Integrated Logic Analyzer	49
6.2	Design Verification	50
6.3	Final Architecture	52

II Development of the Customized RISC-V F extension to support floating-point instructions and arithmetic operations.	53
7 RISC V Floating Point ISA.	54
7.1 Overview	54
7.2 Customized Floating-Point ISA	55
7.3 Rationale for the Selected Instruction Subset	56
8 RISC-V Toolchain	57
8.1 Overview	57
8.2 Compiler	57
8.3 RISC V Modularity	58
8.4 RISC-V Open Source Toolchain	58
8.5 RISC V Simulator (SPIKE)	59
8.6 RISC-V Compilation and Toolchain Flow	59
8.7 Functional Verification	61
8.8 Instruction-Level Representation of Test Programs	61
8.8.1 Assembly-Level Implementation without the F Extension . . .	62
8.8.2 Assembly Level Code with F extension	63
III Integration and Verification of the FPU within the NOVA-Core RISC-V Processor Architecture	66
9 NovaCore In Order Processor Architecture	67
9.1 The NOVACore	67
9.2 Features	68
9.3 Peripherals	68
10 Integration of the Floating-Point Unit into NOVACore	70
10.1 Overview of the Integration	70
10.2 Floating-Point Register File	70
10.3 Decode Stage Modifications	71
10.4 Execute Stage Modifications	72
10.5 Memory and Writeback Stages Modifications	74
10.6 Pipeline Control and Hazard Handling	74
10.6.1 Division Stall Logic	74
10.6.2 RAW Hazards	74
10.6.3 Forwarding Unit	75
10.6.4 Pipeline Stalling Mechanism	75
10.7 Final Architecture	76

11 Results and Discussion	77
11.1 Simulation Based Verification	77
11.1.1 Verification I	77
11.1.2 Verification II	78
11.2 FPGA Verification Results	80
11.3 Observed Limitations	80
12 Conclusion and Future Work	82
12.1 Summary of Contributions	82
12.2 Future Improvements	83
Appendix	88
A Extra Data Supporting the Architecture	88
Bibliography	92

Chapter 1

Introduction

1.1 Background

The RISC-V instruction set architecture, an open-source instruction set architecture (ISA), has received widespread attention due to its extensibility and flexibility, allowing numerous customizations to be made for meeting particular hardware and application needs. Although it has numerous benefits, the RISC-V base instruction set does not provide the optimization required for floating-point arithmetic, a function critical for computationally demanding applications in fields like scientific computing, machine learning, and digital signal processing. [1]

The objective of this thesis is to design and implement a RISC-V compliant Floating-Point Unit (FPU) that adheres to the IEEE 754 standard for single-precision arithmetic. With its clean-slate design, the RISC-V architecture offers several advantages, including openness, simplicity, modularity, extensibility, and long-term stability. The RISC-V core implements a classic RISC pipeline consisting of multiple stages that process instructions in parallel. To improve system performance, all arithmetic units are organized into a three-stage pipeline structure. This pipelined design allows multiple floating-point operations to be processed concurrently, thereby increasing the maximum achievable clock frequency and improving the overall throughput of the processor.

1.2 Motivation

For efficient operation of the arithmetic units, pipelining is an effective design approach. According to previous implementations, the arithmetic units in the processor were designed as combinational modules. While combinational addition, subtraction, and multiplication can complete their computations within only a few cycles,

combinational division requires more than 25 cycles to finish. This long latency significantly limits the overall performance of the processor. Therefore, introducing a pipelined architecture for the floating-point arithmetic units can improve throughput and help achieve better processor performance.

1.3 Problem Statement

As discussed in Section 3.1, the initial floating-point architecture suffered significant timing limitations due to the presence of fully combinatorial arithmetic units within the execution stage. In particular, the division operation introduced a long combinational path, which negatively affected the overall processor performance.

To address these issues, the following objectives are defined:

Objectives

- Verify the existing floating-point design through implementation on an FPGA platform.
- Design a pipelined architecture for the floating-point arithmetic units in order to improve timing performance and throughput.
- Develop an optimized division module to reduce the long combinational path associated with the division operation.
- Implement the redesigned architecture on the FPGA and evaluate the resulting performance improvements.
- Integrate the verified floating-point unit with the NOVACore in-order RISC-V processor.
- Perform comprehensive verification to ensure the correct functionality of the integrated system.

1.4 Development Environment

The implementation of the floating-point unit was carried out using a comprehensive development environment consisting of high-level modeling tools, RTL design and synthesis tools, verification frameworks, and supporting development infrastructure.

1.4.1 High-Level Modeling Environment

- **MSYS2 MINGW64:** Used as the development environment for building and running open-source tools and compilers on Windows, providing a Unix-like shell and package manager for installing required libraries and toolchains.
- **C++ Compiler (GCC 9.3.0):** Utilized for compiling and linking SystemC models with support for the C++14 standard.
- **Build System (CMake 3.16):** Employed to manage cross-platform build configuration and dependency handling.

1.4.2 RTL Design and Synthesis Tools

- **Xilinx Vivado 2019.2:** Used as the primary tool for RTL synthesis, implementation, and FPGA targeting.
- **Hardware Languages:**
 - **System Verilog :** Floating Point Arithmetic modules were designed using System Verilog.
 - **VHDL :** Used to design other floating point operations and the consequent integration to the NovaCore.

1.4.3 Verification and Analysis Tools

- **RISC-V GNU Toolchain:** Provides the cross-compiler and assembler used to generate test programs.
- **Vivado Simulator:** Used as a reference model for functional verification and comparison.
- **Software Languages::**
 - **C :** Used as a reference model to verify floating point results.
 - **Python :** Used to automate creation of LUTs, reference model to verify floating point results.
 - **Shell Scripting:** Used to automate the compilation flow and generation of binary and memory initialization files.

1.4.4 Development Infrastructure

- **Documentation:** Technical documentation prepared using Latex with the TeXstudio editor.
- **Hardware Platform:** Xilinx Digilent Zed evaluation board used for FPGA implementation and validation.

1.5 Development Stages

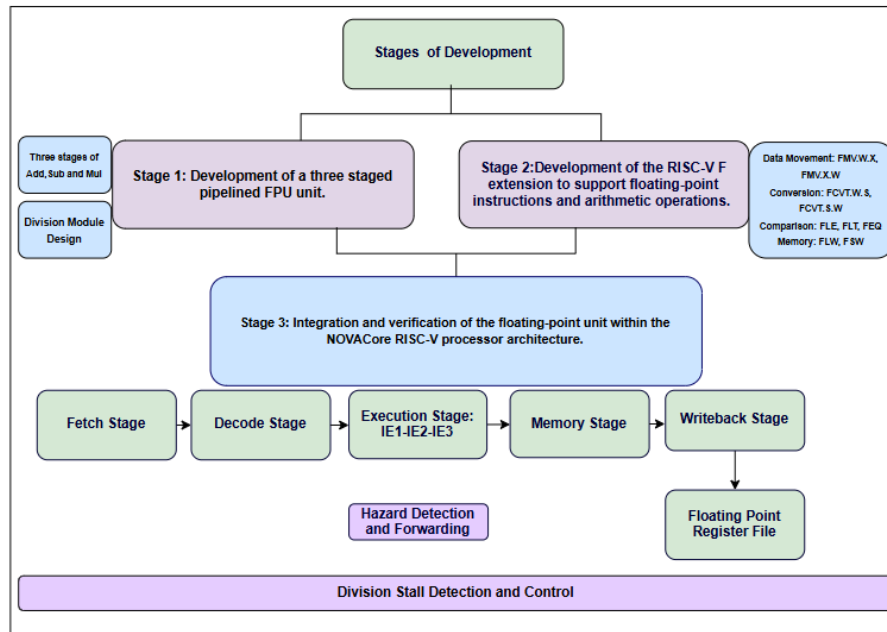


Figure 1.1 – Stages of Development

This thesis is divided into three main parts. The first part focuses on the design and implementation of a pipelined IEEE 754 compliant floating-point arithmetic unit. The second part describes the development of the RISC-V F extension to support floating-point instructions. The final part presents the integration and verification of the floating-point unit within the NOVACore RISC-V processor architecture.

1. Design and implementation of a pipelined IEEE 754 compliant floating-point arithmetic unit for a RISC-V processor.

- **Pipeline Design of Basic Arithmetic Units**

The floating-point addition, subtraction, and multiplication units were implemented using a three-stage pipeline structure in order to improve timing performance and increase the achievable clock frequency.

- **Division Unit Development**

A dedicated floating-point division module was developed to replace the initial combinational implementation. This allowed improved scalability and better timing characteristics.

- **FPGA Implementation and Verification**

The arithmetic units were synthesized and implemented on the FPGA platform to verify correct functionality and timing behavior.

2. Development of the RISC-V F extension to support floating-point instructions and arithmetic operations.

Support for floating-point instructions was implemented, including:

- **Memory operations:** FLW, FSW
- **Conversion operations:** FCVT*
- **Comparison operations:** FLT, FLE, FEQ
- **Movement operations:** FMV*
- **Minimum–Maximum operations:** FMIN, FMAX
- **Arithmetic operations:** FADD, FMUL, FDIV, FSUB

3. Integration and verification of the floating-point unit within the NOVACore RISC-V processor architecture.

- **Development of a Seven-Stage Floating-Point Pipeline**

A seven-stage RISC-V floating-point pipeline was designed consisting of the stages: Fetch, Decode, IE1, IE2, IE3, Memory, and Writeback. A 32-bit floating-point register file was also implemented.

- **Hazard Detection and Pipeline Control**

Pipeline control mechanisms were developed, including RAW hazard detection, load hazard detection, division stall logic, and data forwarding to maintain correct instruction execution.

- **System Verification**

The complete floating-point pipeline and control logic were verified through simulation using various floating-point test programs.

- **Integration with NOVACore RISC-V Processor**

Finally, the floating-point unit was successfully integrated into the NOVACore in-order RISC-V processor architecture.

1.6 Literature Review

The literature review sets the theoretical foundation for this thesis topic and mentions concepts for floating point arithmetic, pipelined architecture, and FPGA implementation. The search uses peer reviewed journals, conference papers, technical standards

(IEEE 754), and reference texts published in the past two decades. Academic databases like IEEE Xplore, ACM Digital Library, Science Direct, and Google Scholar, employing keywords related to floating-point hardware, pipelined processors, SystemC modeling, and FPGA implementation. Floating point representation allows computers to work with real numbers, addressing limitations of fixed arithmetic where a fractional is used for storing a certain fixed number of digits. [2] Floating-point computation is a fundamental requirement in many modern applications, motivating extensive research into the implementation of IEEE 754 compliant floating-point units (FPUs) within RISC-V based systems on FPGA platforms. The IEEE 754 standard was established in 1985 and revised twice in 2008 and 2019 to account for changes and improvements suggested by scholars.

The RISC-V ISA has emerged as a leading instruction set architecture for Internet-of-Things (IoT) and System-on-Chip (SoC) applications. Due to its open-source nature and strong security features, it is increasingly considered a viable alternative to established architectures such as ARM [3] [4]. The work presented in [3] describes the design methodology of a lightweight, open-source RISC-V processor implementing a reduced RV64I instruction set, verified and deployed on an FPGA. In contrast, this thesis focuses on the implementation of floating-point support within a RISC-V architecture. The work in [5] presents the design and implementation of a RISC-V and IEEE 754 compliant Floating-Point Unit (FPU). The results demonstrate that integrating an FPU significantly increases hardware complexity and area, with the proposed design requiring considerably more gate equivalents than minimal RISC-V cores such as Ibex and Snitch. However, this increase in area is justified by the substantial performance improvements achieved through hardware acceleration of floating-point operations, reducing the reliance on software-based implementations. This highlights the fundamental trade-off between area and performance in FPU design.

In [6], a Posit Vector Arithmetic Unit (PVU) is proposed as an alternative to traditional IEEE 754 floating-point systems, addressing limitations in energy efficiency and numerical accuracy. Unlike prior work focused on scalar computation, the PVU supports vector operations such as addition, multiplication, division, and dot products, enabling efficient parallel processing. The design, implemented in Chisel, is modular and parameterizable, achieving high accuracy with relatively low hardware cost. This approach represents a promising research direction, and future extensions of this thesis could explore similar alternative number representations.

The work presented in [7] introduces Faust, a RISC-V compliant FPU adhering to the IEEE 754–2019 standard and integrated within a vector processing accelerator supporting the RISC-V vector extension. The design achieves high performance through a multi-lane architecture and a 5-stage pipeline, reaching frequencies up to 1.82 GHz in synthesis. While Faust demonstrates excellent throughput and scalability

for vectorized workloads, its complexity and area overhead make it more suitable for high-performance systems rather than resource-constrained FPGA implementations. Floating-point units function as specialized coprocessors designed to efficiently execute arithmetic operations such as addition, subtraction, multiplication, and division, particularly in computation-intensive domains like signal processing [8]. In [8], both high-performance and area-optimized variants of a floating-point coprocessor were developed for the Vega processor using Bluespec SystemVerilog (BSV). These designs support a wide range of RISC-V floating-point instructions, including arithmetic, comparison, conversion, and classification operations, across both single and double precision formats.

The paper [9] proposes a modular FPU microarchitecture for embedded systems that optimizes the trade-off between energy efficiency, accuracy, and hardware area by allowing configurable precision for floating-point operations. Experimental results demonstrate improved energy-delay product and reduced resource utilization with minimal accuracy loss, highlighting the effectiveness of mixed-precision designs.

In contrast to the aforementioned works, this thesis presents a pipelined implementation of a reduced subset of the RV32F instruction set on the NOVACore processor, along with its verification on an FPGA platform [10]. The individual arithmetic operations are implemented in a pipelined manner, and division algorithms have been extensively studied [11]. A comprehensive analysis of these approaches, including their advantages and limitations, is also provided.

Chapter 2

Architecture Overview

2.1 RISC-V Architecture

2.1.1 Pipelining

A pipeline is a set of data processing elements connected in series, where the output of one element is the input of the next one.

Single-cycle processors execute one complete instruction per clock cycle, making them simple but slow, as the clock must accommodate the slowest instruction. [12]

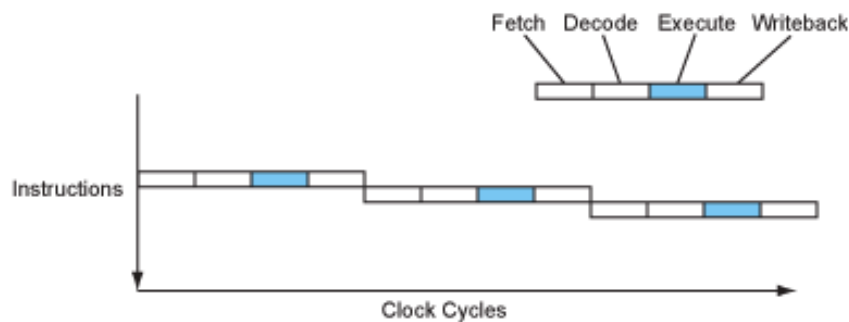


Figure 2.1 – Single Cycle Processing
[12]

Pipelined processors overlap instruction stages (fetch, decode, execute) to achieve higher throughput, completing one instruction per cycle, but introducing complexity via hazards.

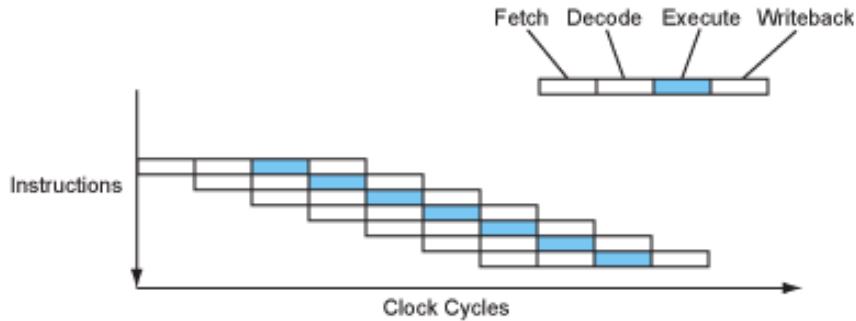


Figure 2.2 – Pipelined Processing
[12]

Five-Stage RISC-V Pipeline

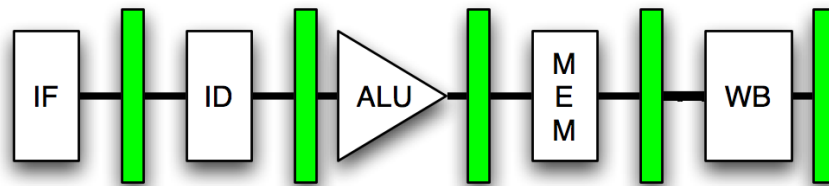


Figure 2.3 – Pipeline Stages
[12]

The processor follows a classical five-stage pipeline architecture consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage performs a specific operation on the instruction as it progresses through the pipeline. [12]

- **Instruction Fetch (IF)**

This stage fetches the next instruction to be executed from the instruction memory. The program counter (PC) provides the address of the instruction, which is then forwarded to the next pipeline stage.

- **Instruction Decode and Register Fetch (ID)**

In this stage, the fetched instruction is decoded to determine the operation to be performed. If required, operand values are read from the register file. The register file allows two registers to be read simultaneously, therefore two output wires are used to forward the register values to the next stage.

- **Execution, Memory Address Computation, or Branch Completion (EX)**

The Arithmetic Logic Unit (ALU) performs operations on up to two operands provided by the previous stage. Depending on the instruction type, the ALU can perform one of the following operations: [12]

- **Memory reference:** The ALU adds operands to compute the effective address for data memory access.
- **Arithmetic-logical instruction:** The ALU performs arithmetic or logical operations specified by the instruction on the two operand values.
- **Branch:** The ALU compares the two register values. The comparison result determines whether a branch should be taken or not.

- **Memory Access (MEM)**

In this stage, instructions that require memory operations access the data memory. A value may be read from or written to the memory address calculated by the ALU in the previous stage. Instructions that do not require memory access simply bypass this stage.

- **Write Back (WB)**

In the final stage, the result of the instruction is written back to the destination register in the register file if required. For example, the result of an arithmetic operation computed by the ALU is stored in the destination register.

Further, in between the stage components there are additional components, so called **pipeline registers**.

2.1.2 In Order Processor

An in-order RISC-V architecture refers to a processor design where instructions are fetched, decoded, executed, and completed (committed) in the exact order specified by the compiler.

This contrasts with out-of-order processors, which can execute instructions in a different order to maximize performance. In-order designs are generally simpler, smaller, and more power-efficient, making them ideal for embedded systems and low-power applications.

Pipeline Hazards

In a pipelined processor, multiple instructions are executed simultaneously in different pipeline stages. However, certain situations may prevent the next instruction from executing in the following clock cycle. These situations are known as *pipeline hazards*. In an in-order pipeline, the following types of hazards may occur. [13] [4]

- **Structural Hazards**

Structural hazards occur when two or more instructions attempt to use the same hardware resource at the same time. For example, if instruction memory and data memory share the same hardware resource, a conflict may arise between the instruction fetch stage and the memory access stage. [13] [4]

- **Data Hazards**

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed its execution. In such cases, the required data may not yet be available when the next instruction attempts to read it. [13] [4]

- **Control Hazards**

Control hazards arise from branch or jump instructions that change the program flow. When a branch instruction is encountered, the processor may not yet know which instruction should be executed next until the branch condition is evaluated. [13] [4]

2.2 RISC-V Floating-Point Extensions

The RISC-V Floating Point Extension (F-extension) is a standard instruction set extension that introduces single-precision floating point operations compliant with the IEEE 754-2008 standard. It is designed to enhance computational capabilities for floating-point arithmetic while maintaining simplicity and efficiency.

2.2.1 Key Features of the F-Extension

- **Floating-Point Registers:** The F-extension adds 32 floating-point registers (f0 to f31), each 32 bits wide, and a Floating-Point Control and Status Register (FCSR). The FCSR manages rounding modes and exception flags. [14]
- **Rounding Modes:** The extension supports multiple rounding modes, including: Round to Nearest, ties to Even (RNE) Round towards Zero (RTZ) Round Down (RDN) Round Up (RUP) Round to Nearest, ties to Max Magnitude (RMM) Dynamic rounding mode (DYN), which uses the value in the FCSR. [14]
- **Arithmetic Operations:** It provides instructions for addition (FADD.S), subtraction (FSUB.S), multiplication (FMUL.S), division (FDIV.S), and square root (FSQRT.S). These operations adhere to IEEE-754 standards. [14]

- **Fused Multiply-Add (FMA):** Instructions like FMADD.S, FMSUB.S, FNMSUB.S, and FNMADD.S perform fused multiply-add/subtract operations, reducing rounding errors and improving performance. [14]
- **Load and Store Instructions:** The FLW and FSW instructions handle floating-point data transfer between memory and registers. They ensure atomic execution for naturally aligned addresses. [14]
- **Conversion Instructions:** The extension includes instructions for converting between floating-point and integer formats, such as FCVT.W.S (float-to-int) and FCVT.S.W (int-to-float). These conversions handle out-of-range values by clipping and setting exception flags. [14]
- **Comparison and Classification:** Instructions like FEQ.S, FLT.S, and FLE.S compare floating-point values, while FCLASS.S classifies floating-point numbers into categories like infinity, NaN, subnormal, and zero. [14]
- **NaN Handling:** The extension uses a canonical NaN representation for invalid operations. It avoids propagating NaN payloads to reduce hardware complexity. [14]
- **Subnormal Arithmetic:** Operations on subnormal numbers comply with IEEE 754-2008, detecting tininess after rounding to minimize spurious underflow signals. [14]

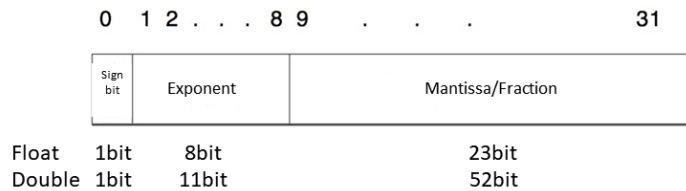
2.3 IEEE 754 Standard

2.3.1 Overview

The IEEE 754 standard is the widely adopted specification for representing and performing arithmetic on floating-point numbers in digital computers. It defines how real numbers are stored, how arithmetic operations behave, and how to handle exceptions like overflow, underflow, and invalid operations.[15] [16]

Real value

▪ IEEE 754 floating-point standard



Normalized format

E=0 and 255 (or 2047) are reserved for special use
(i.e. 1~254 (or 2046) are used for normalized floating point numbers)

$$(-1)^S \times (1.M) \times 2^{E-127} \quad (\text{for float type})$$

$$(-1)^S \times (1.M) \times 2^{E-1023} \quad (\text{for double type})$$

S: sign bit, **M:** Mantissa, **E:** Exponent

Figure 2.4 – IEEE 754 Floating-Point 32 Bit Single Precision Layout
[17]

Representation

A floating-point number has three main components:

- **Sign bit (s):** Determines positivity (0) or negativity (1).
- **Exponent (e):** Encodes the scale (magnitude) of the number, stored with a bias to support both positive and negative exponents.
- **Significand / Mantissa (m):** Represents the precision digits of the number, usually normalized such that the leading digit is 1 (in binary).

The general formula is:

$$(-1)^s \times 1.m \times 2^{(e-\text{bias})}$$

2.3.2 Standard Formats

Single Precision (32-bit)

- **Configuration:** 1-bit sign, 8-bit exponent, 23-bit mantissa.

- **Exponent Bias:** 127.
- **Precision:** Approximately 7 decimal digits.
- **Range:** Roughly $\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$.

Double Precision (64-bit)

- **Configuration:** 1-bit sign, 11-bit exponent, 52-bit mantissa.
- **Exponent Bias:** 1023.
- **Precision:** Approximately 15–17 decimal digits.
- **Range:** Roughly $\pm 2.23 \times 10^{-308}$ to $\pm 1.8 \times 10^{308}$.

Extended Precision

Some architectures provide extended precision (80-bit or higher) for intermediate calculations to reduce rounding errors.

2.3.3 Special Values

IEEE 754 defines specific bit patterns for exceptional cases:

- **Zero:** Exponent = 0, Mantissa = 0.
- **Infinity (∞):** Exponent = all 1s, Mantissa = 0. Used for overflow or division by zero.
- **NaN (Not a Number):** Exponent = all 1s, Mantissa $\neq$ 0. Used for invalid operations (e.g., $0/0$).
- **Denormalized Numbers:** For very small magnitudes; exponent = 0, mantissa is interpreted without the leading 1 to represent gradual underflow.

2.3.4 Rounding Modes

IEEE specifies several rounding strategies to maintain numerical stability:

- **Round to Nearest, ties to even** (Default)
- **Round toward zero**
- **Round toward $+\infty$**
- **Round toward $-\infty$**

The work presented in this thesis is based on 32-bit single-precision IEEE 754 floating-point representation.

2.4 Field Programmable Gate Array

A Field-Programmable Gate Array (FPGA) is a reprogrammable integrated circuit consisting of configurable logic blocks and programmable interconnects, allowing hardware functionality to be defined or updated after manufacturing using Hardware Level languages like VHDL or (System)Verilog. [18]

2.4.1 FPGA Architecture Overview

An FPGA consists of several key components that enable flexible and reconfigurable hardware design[18]:

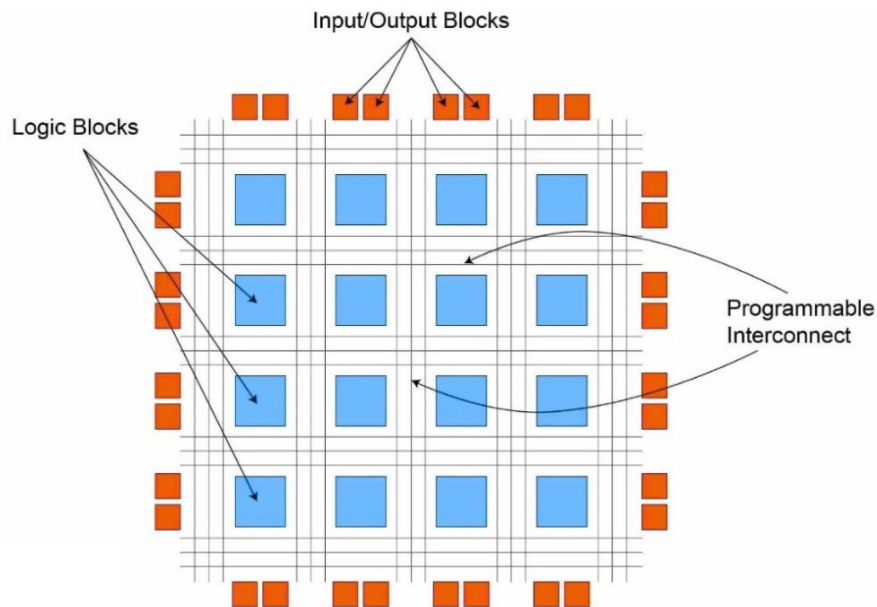


Figure 2.5 – FPGA Architecture
[19]

1. Logic Blocks:

Logic blocks form the fundamental building units of an FPGA. Each block contains combinational logic elements and storage elements such as registers. The combinational logic is used to implement Boolean functions, while the registers store intermediate and sequential data.

2. Interconnect Network:

The interconnect network provides programmable routing resources that con-

nect the logic blocks. This reconfigurable routing enables the implementation of custom data paths and control logic tailored to specific applications.

3. Configuration Memory:

The configuration memory stores the programming data that defines the functionality of the FPGA. This data is loaded during device configuration, allowing the FPGA to realize the desired hardware behavior.

2.4.2 Key Characteristics and Advantages:

- **Reprogrammability:** Unlike ASICs (Application-Specific Integrated Circuits), FPGAs can be reconfigured repeatedly, allowing for easy updates and upgrades.
- **Parallel Processing:** FPGAs handle tasks simultaneously, providing superior performance over CPUs for specific parallel algorithms.
- **Short Time-to-Market:** Because they are not hardwired during fabrication, FPGA development allows for faster prototyping and deployment compared to ASIC design.
- **Flexibility and Performance:** They provide a balance between the efficiency of hardware and the flexibility of software. [18]

Chapter 3

Analysis of the Initial Floating-Point Architecture

This chapter presents the baseline floating-point architecture and evaluates its performance on FPGA. The analysis highlights key timing limitations, particularly due to the combinational arithmetic units, which motivate the need for a pipelined redesign.

3.1 Initial System Architecture

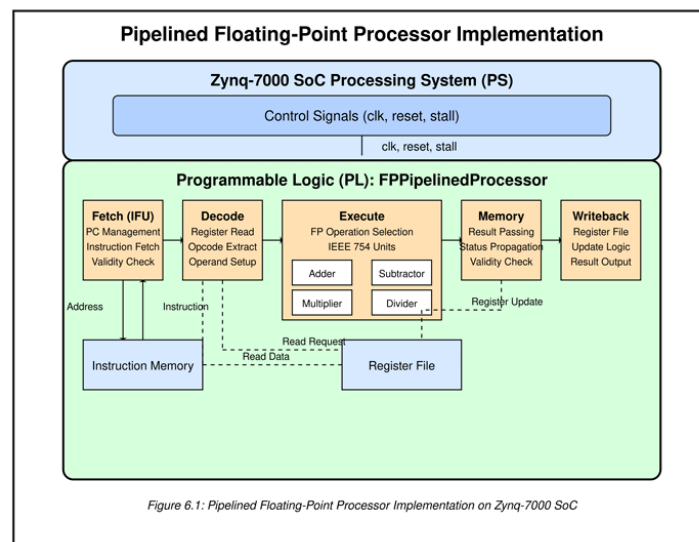


Figure 3.1 – Initial Architecture

[2]

As illustrated in Figure 3.1, the current architecture features a pipelined floating-point processor.

- The five-stage pipeline gives an effective balance between throughput and implementation ease. Separating the IEEE 754 operations across stages enables simpler handling of arithmetic calculations.
- The floating-point unit's preparation for implementation on FPGA platforms has been conveniently achieved through RTL implementation and synthesis preparation. It is a solid start, but more testing needs to be done on a variety of FPGA platforms.
- Early detection and processing of IEEE 754 special cases (zero, infinity, NaN) throughout pipeline significantly reduced errors.
- The entire system is designed so that each instruction is executed in one clock cycle (no pipelining).

The design was initially verified through simulation. The subsequent hardware implementation on the FPGA platform is described in Section 3.2.

3.2 FPGA-Based Processor Implementation

The proposed architecture was synthesized and implemented on a ZedBoard FPGA platform. During the implementation phase, the timing performance of the design was evaluated using the timing analysis tools provided in Vivado.

3.2.1 Zed board

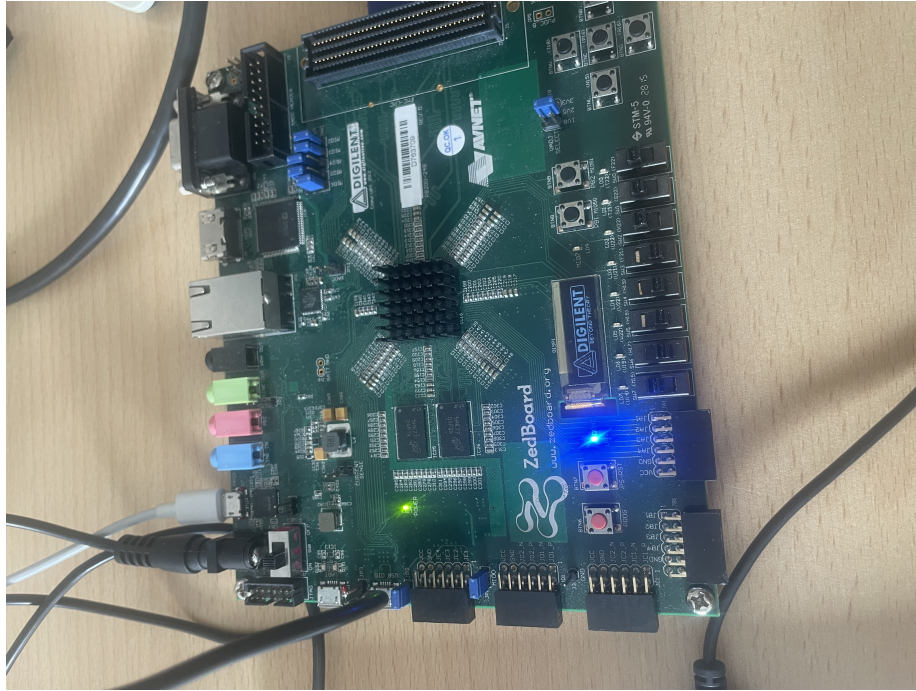


Figure 3.2 – Zed Board

ZedBoard is a low-cost development board for the AMD Zynq™ 7000 all programmable SoC (AP SoC). This board contains everything necessary to create a Linux, Android, Windows, or other OS/RTOS based design. Additionally, several expansion connectors expose the processing system and programmable logic I/Os for easy user access. The ZedBoard leverages the AMD Zynq™-7000 All Programmable SoC, which tightly integrates a dual-core ARM® processing system with 7-series programmable logic. This architecture enables efficient hardware–software co-design, allowing the development of complex and high-performance embedded systems. [20] The platform supports a wide range of applications, including video processing, motor control, software acceleration, and embedded system development using operating systems such as Linux, Android, and real-time operating systems (RTOS). Additionally, it serves as a versatile platform for prototyping and evaluating designs based on the Zynq-7000 SoC.

Category	Specification
Core Features	
FPGA Part #	XC7Z020-CLG484
Programming	USB-JTAG (Digilent SMT1-equivalent)
Clock Sources	
Oscillators	33.333 MHz (PS), 100 MHz (PL)
Communication	
Ethernet	10/100/1000 Mbps
USB	USB OTG 2.0
SD Card	4 GB
Connectivity and I/O	
Pmod Connectors	5 (2×6)
HDMI	HDMI Output
VGA	12-bit VGA
Display	128×32 OLED
Audio	Line-in, Line-out, Headphone, Microphone
FMC	1 LPC FMC
AMS	1 AMS Header
User Interface	
Switches	8 DIP/Slide Switches
Push-buttons	7
Reset Buttons	2
User LEDs	9
DONE LED	1
Electrical	
Power Supply	12V, 5A AC/DC
Logic Level	3.3V
Physical	
Dimensions	6.3 in × 6.3 in
Compliance	
HTC Code	8471500150
ECCN	5A992.c

[20]

Table 3.1 – ZedBoard (Zynq-7000 Development Board) Specifications

3.2.2 Test Setup

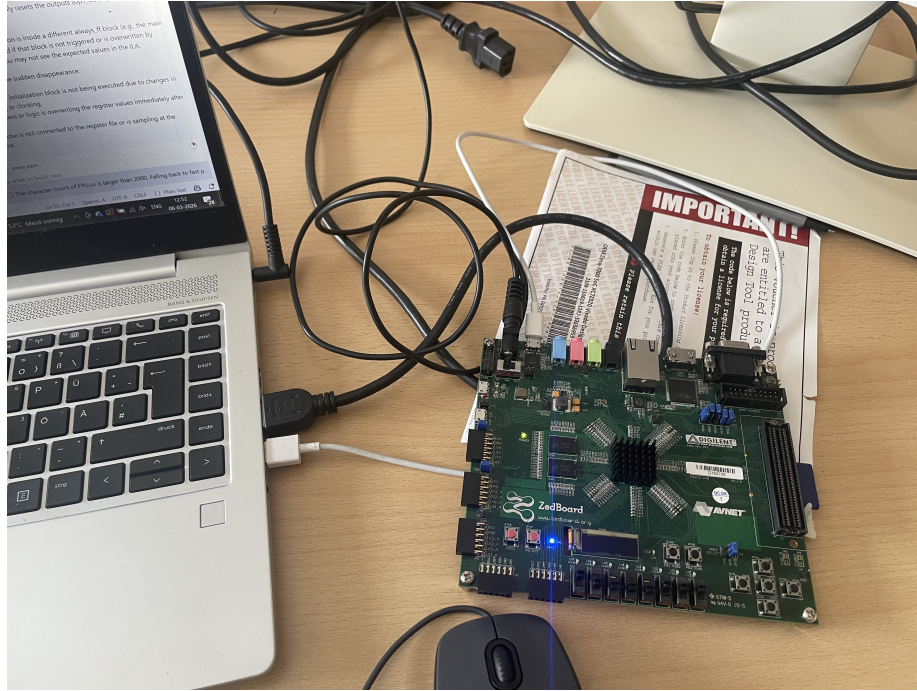


Figure 3.3 – Test Setup.

The experimental setup used for validating the proposed design is shown in Figure 3.3. The implementation is carried out on the ZedBoard, which hosts the Zynq-7000 SoC. The board is connected to the host system using a JTAG interface for programming and debugging.

The FPGA is configured using Xilinx Vivado Hardware Manager. The device is detected by Hardware manager and the generated bitstream file (.bit) and a corresponding probe file (.ltx) are programmed into the device.

The probe file contains the definitions of internal signals to be monitored. This enables visibility into internal pipeline signals and control paths during execution. For debugging and real-time signal observation, the Integrated Logic Analyzer (ILA) is utilized. The ILA allows capturing and analyzing signal activity directly on the FPGA, facilitating verification of the processor functionality and aiding in debugging of the floating-point unit (FPU) operations.

The overall setup enables efficient hardware validation of the design through JTAG-based programming and on-chip debugging using ILA.

- To address the timing issue, the Xilinx Clocking Wizard IP was used to generate a lower-frequency clock for the design.
- After reducing the clock frequency to 5 MHz, the timing constraints were successfully satisfied.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology
✓ synth_1 (active)	constrs_1	synth_design Complete!									
✓ impl_1	constrs_1	write_bitstream Complete!	24.865	0.000	0.038	0.000	32.114	0.000	0.206	0	8 Warn
Out-of-Context Module Runs											
✓ ila_0_synth_1	ila_0	synth_design Complete!									
✓ clk_wiz_0_synth_1	clk_wiz_0	synth_design Complete!									

Figure 3.6 – Timing analysis showing successful timing closure at 5 MHz.

- The design was able to operate reliably at a maximum clock frequency of approximately **9 MHz**.
- The **critical path** of the design was identified within the **combinational division module**, which introduced a long propagation delay and significantly affected the timing performance of the processor. The critical path was identified using the command

```
report_timing -nworst 5 -path_type full -input_pins -file timing_paths.rpt.
```
- The resulting low operating frequency was undesirable for practical applications. Therefore, a pipelined floating-point unit architecture was proposed in order to improve timing performance and increase the achievable clock frequency.

Conclusion

A significant bottleneck exists because all four arithmetic units are located within the execution stage. Due to their combinational nature, these units create a lengthy critical path that constrains the maximum clock frequency, ultimately limiting the overall throughput of the system.

Part I

Design of a Pipelined IEEE 754 Compliant Floating-Point Arithmetic Unit for a RISC-V Processor

Chapter 4

Floating-Point Arithmetic Unit Design

As shown in Section 3.2, the combinational implementation results in a critical timing bottleneck. To address this issue, the design is restructured using a pipelined architecture.

4.1 Overview of Arithmetic Units

4.1.1 Methodology

Each component of the arithmetic units is implemented as a System Verilog module with a focus on the code being conventional and highly readable. The instruction memory is realized as a read-only memory (ROM) and is initialized with program instructions at the time of FPGA configuration. The memory size is configurable through parameters in the SystemVerilog design. Each memory entry is 32 bits wide, corresponding to a single 32-bit RISC-V instruction. Initialization is performed via a compiler-generated “.mem” file containing space-separated instruction values. [3]

4.1.2 Special Case Handling

IEEE reserves exponent field values of all 0s and all 1s to denote special values in the floating-point scheme. [17]. [15]

```
1  ' ' '
2  a_nan = (exp_a == 8'hFF) && ((a & 32'h7FFFFFFF) != 0);
3  b_nan = (exp_b == 8'hFF) && ((b & 32'h7FFFFFFF) != 0);
4  a_inf = (exp_a == 8'hFF) && ((a & 32'h7FFFFFFF) == 0);
5  b_inf = (exp_b == 8'hFF) && ((b & 32'h7FFFFFFF) == 0);
6  ' ' '

```

Listing 4.1 – Detection of Special IEEE 754 Floating-Point Cases

The code snippets provide insight into the handling of special cases within the design. The FPU effectively implements IEEE 754 compliant behavior, including normalization, rounding, overflow, and underflow handling.

```

1
2 // Check overflow/underflow
3 if (exp_local >= 8'hFF) begin
4 result = {sign_local, 8'hFF, 23'd0};
5 temp_flags[2] = 1; // Overflow flag
6 temp_flags[0] = 1; // Inexact flag
7 end else if (exp_local <= 0) begin
8 result = {sign_local, 8'd0, 23'd0};
9 temp_flags[1] = 1; // Underflow flag
10 temp_flags[0] = 1; // Inexact flag
11 end

```

Listing 4.2 – Overflow and Underflow Detection Logic

4.1.3 Rounding Modes

The different rounding modes explained in Section 2.3.4 have been implemented in the arithmetic modules. Listing 4.3 describes the rounding modes in the subtraction module. All modules are properly equipped with proper implementations of the rounding modes.

```

1
2 ‘‘‘
3 // Round according to mode
4 case (rm)
5   3'b000:
6     if (guard && (round || sticky || norm_mant[3]))
7       mant_rounded_wide = {1'b0, norm_mant[26:3]} + 25'd1;
8     else
9       mant_rounded_wide = {1'b0, norm_mant[26:3]};
10
11   3'b001:
12     mant_rounded_wide = {1'b0, norm_mant[26:3]};
13
14   3'b010:
15     if (in_sign && (guard || round || sticky))
16       mant_rounded_wide = {1'b0, norm_mant[26:3]} + 25'd1;
17     else
18       mant_rounded_wide = {1'b0, norm_mant[26:3]};
19
20   3'b011:
21     if (!in_sign && (guard || round || sticky))
22       mant_rounded_wide = {1'b0, norm_mant[26:3]} + 25'd1;
23     else

```

```

24         mant_rounded_wide = {1'b0, norm_mant[26:3]};
25
26     3'b100:
27         if (guard)
28             mant_rounded_wide = {1'b0, norm_mant[26:3]} + 25'd1;
29         else
30             mant_rounded_wide = {1'b0, norm_mant[26:3]};
31
32     default:
33         if (guard && (round || sticky || norm_mant[3]))
34             mant_rounded_wide = {1'b0, norm_mant[26:3]} + 25'd1;
35         else
36             mant_rounded_wide = {1'b0, norm_mant[26:3]};
37 endcase
38 '''

```

Listing 4.3 – Rounding Logic Based on IEEE 754 Rounding Modes

Along with the rounding modes , the different flags are also implemented.

```

1
2 '''
3 // Flags
4 flags[4] = (norm_exp == 8'hFF) && (norm_mant[25:3] != 23'd0); \
// NV (Invalid)
5 flags[3] = 1'b0; \
// DZ (Divide by Zero)
6 flags[2] = (result_temp[30:23] == 8'hFF) && \
// OF (Overflow)
// (result_temp[22:0] == 23'd0); \
7 flags[1] = (result_temp[30:23] == 8'd0) && \
// UF (Underflow)
// (result_temp[22:0] != 23'd0); \
8 flags[0] = guard | round | sticky; \
// NX (Inexact)
9
10
11 '''

```

Listing 4.4 – IEEE 754 Exception Flag Generation

The implementation of the Floating-Point Control and Status Register (FCSR) relies on the correct handling of rounding modes and exception flags, which collectively govern the rounding behavior of floating-point operations and capture exceptional events such as overflow, underflow, and invalid operations.

4.1.4 Guard, Round, and Sticky Bits

In floating-point arithmetic, intermediate results often exceed the precision that can be represented within the fixed mantissa width defined by the IEEE 754 standard.

To ensure correct rounding of such results, three additional bits—namely the *guard*, *round*, and *sticky* (GRS) bits—are used during computation. [21]

The **guard bit** is the first bit beyond the least significant bit (LSB) of the normalized mantissa. It provides information about the immediate next bit that would be truncated during rounding. The **round bit** is the bit following the guard bit and further refines the rounding decision. The **sticky bit** represents the logical OR of all remaining lower-order bits beyond the round bit, indicating whether any non-zero bits were discarded during alignment or shifting operations.

These bits play a critical role in implementing IEEE 754 rounding modes. For example, in the commonly used round-to-nearest-even mode, the guard, round, and sticky bits are used together to determine whether the mantissa should be incremented. Specifically, rounding occurs when the guard bit is set and at least one of the following conditions is true: the round bit is set, the sticky bit is set, or the least significant bit of the mantissa is one (to ensure even rounding).

In the proposed FPU design, the mantissa is extended to include these three additional bits during intermediate stages of computation. This allows accurate rounding decisions to be made before truncating the result back to the standard mantissa width. The use of GRS bits ensures compliance with IEEE 754 rounding requirements while maintaining numerical accuracy.

4.2 Three-Stage Pipelined Architecture

4.2.1 Execute Stage Design

The Execute (EX) stage is implemented as a three-cycle pipeline. Accordingly, all floating-point arithmetic units — addition, subtraction, multiplication, and division — are structured to operate across three pipeline stages to maintain timing consistency within the processor.

The arithmetic units are partitioned into the following three stages:

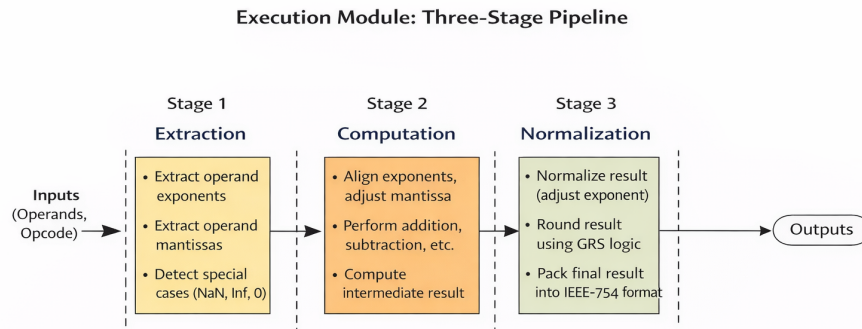


Figure 4.1 – Execution Stage

- **Stage 1: Extraction**

In this stage, the sign, exponent, and mantissa fields are extracted from the IEEE-754 single-precision operands. Special cases such as zero, infinity, and NaN are also detected.

- **Stage 2: Computation**

The core arithmetic operation is performed. This includes exponent alignment and mantissa addition/subtraction for ADD/SUB, and exponent addition with mantissa multiplication for MUL. In the initial pipelined implementation, the computation stage of each arithmetic unit remained combinational. Specifically, the ADD, SUB, MUL, and DIV operations were realized using the built-in arithmetic operators (+, −, ×, /), resulting in a large combinational delay within the execution stage.

- **Stage 3: Normalization and Rounding**

The intermediate result is normalized to restore the IEEE-754 format. Rounding is applied as required, and the final sign, exponent, and mantissa fields are assembled to produce the result.

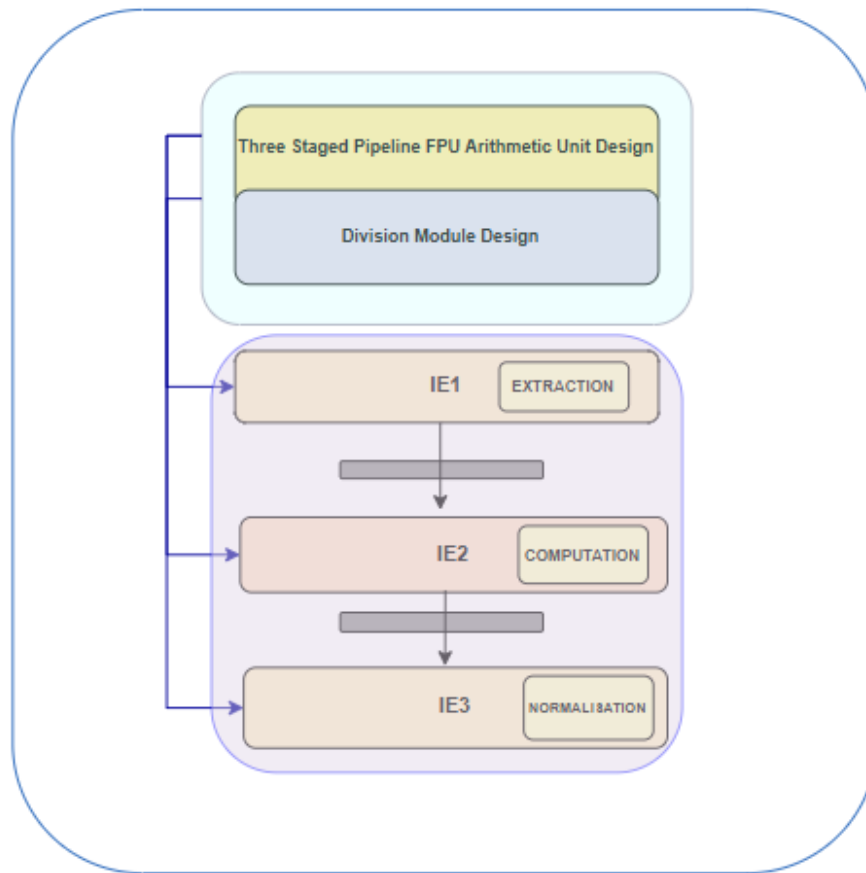


Figure 4.2 – Three Staged Execution Stage

4.2.2 Results

The redesigned architecture was simulated in Vivado using the existing testbench, and the results were consistent with those obtained from the previous implementation.

Subsequently, the pipelined architecture was implemented on the ZedBoard FPGA platform with a target clock frequency of 100 MHz.

Timing Analysis Summary

Although the design produced correct computational results, static timing analysis revealed that the maximum achievable clock frequency remained limited to approximately 11 MHz. Consequently, the introduction of pipelining did not yield the expected performance improvement which is expected since the modules are still combinatorial.

Timing analysis indicated that the critical path traversed the floating-point division module.

This excessive combinatorial delay of the division dominated the overall timing path and constrained the maximum achievable clock frequency of the processor.

4.2.3 Division Architecture Alternatives

In order to improve the overall performance of the architecture, some alternative floating-point division modules were researched.

4.2.3.1 Various Division Algorithms

Researchers have proposed many algorithms and procedural architectures to carry out division in order to reduce the computational time and thus enhancing the performance.

- **Restoring Division:** Restoring division is a binary division algorithm that operates on fixed-point fractional numbers. It is based on the following assumptions: (i) $D < N$ and (ii) $0 < N, D < 1$. The quotient digits q are selected from the set $\{0, 1\}$.

To compute $\frac{a}{b}$, the dividend a is stored in register A , the divisor b in register B , and register P is initialized to zero. [22]

The radix-2 restoring division algorithm is described as follows:

1. Shift the register pair (P, A) one bit to the left.
2. Subtract the contents of register B from register P .
3. If the result of Step 2 is negative, set the least significant bit A_0 to 0; otherwise, set it to 1.
4. If the result of Step 2 is negative, restore the previous value of P by adding the contents of register B back to P .

After repeating the above steps n times, register A contains the quotient, while register P holds the remainder. [22]

- **Non-Restoring Division :** Non-restoring division is a binary division algorithm that utilizes the digit set $\{-1, 1\}$ for quotient generation, unlike restoring division which uses $\{0, 1\}$. This approach eliminates the need for explicit restoration in every iteration, thereby improving efficiency.

To compute $\frac{a}{b}$, the dividend a is stored in register A , the divisor b in register B , and register P is initialized appropriately.

The radix-2 non-restoring division algorithm is described as follows:

1. If P is negative:

- (a) Shift the register pair (P, A) one bit to the left.
 - (b) Add the contents of register B to register P .
2. Else (if P is non-negative):
 - (a) Shift the register pair (P, A) one bit to the left.
 - (b) Subtract the contents of register B from register P .
3. If P is negative, set the least significant bit A_0 to 0; otherwise, set it to 1.

The above steps are repeated for n iterations. After n cycles, register A contains the quotient. If P is positive, it represents the remainder; otherwise, it must be restored by adding the contents of register B to obtain the correct remainder. [22]

- SRT Division: Named for its creators (Sweeney, Robertson, and Tocher), SRT division is a popular method for division in many microprocessor implementations. SRT division is similar to non-restoring division, but it uses a lookup table based on the dividend and the divisor to determine each quotient digit. [22]
- Vedic Architecture: A new approach is the Vedic Sutra, Nikhilam which can be further extended to carry out Binary Division as an alternative to conventional algorithm which is described in [22].
- Newton Raphson Approximate Method: An approximate division algorithm is described in [11]–[23] which is based on Newton-Raphson's iterations.

Floating-point division can be realized using multiple algorithmic approaches. The following sections present and implement two such methods.

- Newton–Raphson Division Module (approximation-based method)
- Restoring iterative Division Module

A detailed description of both architectures is provided in 5.

Chapter 5

Floating-Point Division Architectures

In the recent times, division is very important for several applications in digital signal and image processing, computer graphics and scientific computing . But compared to other floating point operations , division is the most time consuming operation .Hence, designing an efficient floating-point division unit is critical for the overall performance of the processor.[23] ,[24] ,[25]

5.1 Newton-Raphson Divider

This section describes the implementation of a hardware-optimized IEEE-754 single-precision floating-point divider using the Newton-Raphson iterative approximation method. The divider computes A / B where both operands are 32-bit IEEE-754 single-precision floating-point numbers, producing results compliant with the IEEE-754 standard including proper handling of special cases (NaN, Infinity, Zero, Denormals).

5.1.1 Algorithm and Architecture

The Newton–Raphson method is used to compute the reciprocal of the divisor. Starting from an initial approximation x_0 , the reciprocal is refined iteratively using

$$x_{n+1} = x_n(2 - D \cdot x_n) \quad (5.1)$$

where D represents the divisor. The final quotient is obtained as

$$Q = N \cdot x_n \quad (5.2)$$

where N is the dividend.

So in other words the Newton-Raphson method is an iterative algorithm that approximates division by computing the reciprocal of the divisor and then multiplying by the dividend:

$$\text{Division: } Q = \frac{A}{B} \quad (5.3)$$

$$\text{Approach: } Q = A \times \left(\frac{1}{B}\right) \quad (5.4)$$

$$\text{where } \left(\frac{1}{B}\right) \approx Y_2 \text{ (computed iteratively)} \quad (5.5)$$

A high-speed reciprocal unit is highly beneficial for division operations, as division can be transformed into a multiplication-based approach. In this method, the reciprocal of the divisor is first computed and then multiplied with the dividend to obtain the result. This approach is especially efficient when multiple divisions are performed with the same divisor, since the reciprocal needs to be computed only once, and subsequent operations require only a single multiplication. [23]

Algorithm Steps

- **Step 1: Initial Approximation**

$$Y_0 = \text{LUT}(B)$$

(The lookup table provides an 11-bit approximation)

- **Step 2: First Newton-Raphson Iteration (NR1)**

$$Y_1 = Y_0 \times (2 - B \times Y_0)$$

- **Step 3: Second Newton-Raphson Iteration (NR2)**

$$Y_2 = Y_1 \times (2 - B \times Y_1)$$

- **Step 4: Final Quotient**

$$Q = A \times Y_2$$

- **Step 5: Normalization and Packing**

$$\text{Result} = \text{Normalize}(Q)$$

(The result is represented in IEEE-754 format)

Architecture

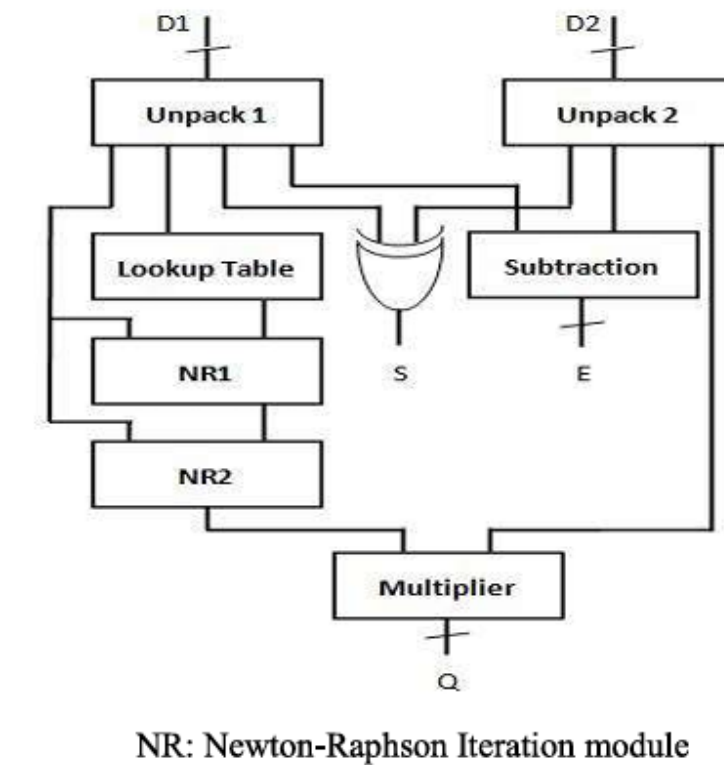


Figure 5.1 – NR Division Architecture

[23]

Convergence Property

Each Newton-Raphson iteration doubles the number of accurate bits:

- Initial LUT : 11 bits accuracy
- After NR1 : 22 bit accuracy
- After NR2 : 24 bit accuracy (sufficient for single precision)

Pipeline Stages of the Newton-Raphson Based Division Unit

- **Stage 1: Operand Extraction**
Extract the sign, exponent, and mantissa from operands A and B . The implicit leading 1 is added for normalized numbers, while denormalized numbers

(exponent = 0) are handled separately.

Latency: 1 cycle

- **Stage 2: Normalization and LUT Approximation**

Leading Zero Counter (LZC):

- Uses a casez-based priority encoder for fast leading zero counting
- Provides constant-time operation compared to iterative methods
- Normalizes the divisor mantissa to 1.xxxx format

LUT Lookup:

- Uses an 11-bit address derived from the normalized mantissa
- Returns an initial reciprocal approximation Y_0
- Provides approximately 11 bits of accuracy

Latency: 1 cycle

- **Stage 3: First Newton-Raphson Iteration (NR1)**

$$Y_1 = Y_0 \times (2 - B \times Y_0)$$

- Compute $product_1 = B \times Y_0$
- Compute $correction_1 = 2 - product_1$
- Compute $Y_1 = Y_0 \times correction_1$

Accuracy: ~ 22 bits

Hardware: DSP48E1 multipliers

Latency: 1 cycle

- **Stage 4: Second Newton-Raphson Iteration (NR2)**

$$Y_2 = Y_1 \times (2 - B \times Y_1)$$

- Compute $product_2 = B \times Y_1$
- Compute $correction_2 = 2 - product_2$
- Compute $Y_2 = Y_1 \times correction_2$

Accuracy: ~ 24 bits (sufficient for single precision)

Hardware: DSP48E1 multipliers

Latency: 1 cycle

- **Stage 5: Quotient Multiplication**

$$Q = A \times Y_2$$

Multiply the dividend by the computed reciprocal.

Latency: 1 cycle

- **Stage 6: Normalization and Packing**

Normalize the result mantissa to 1.xxxx format and adjust the exponent accordingly. The final result is packed into IEEE-754 format:
 $\{\text{sign}, \text{exponent}[7 : 0], \text{mantissa}[22 : 0]\}$

Overflow and underflow conditions are also handled.

Latency: 1 cycle

Sequential FSM Implementation of NR Stages

The Newton–Raphson division can also be controlled by a finite state machine (FSM) in Stage 2, which sequentially executes the reciprocal approximation and refinement steps. The FSM transitions through six states: IDLE, LUT, NR1, NR2, MULTIPLY, and DONE. This will introduce a latency of 7 clock cycles, during which the pipeline is stalled.

5.1.2 Verification Results

The division module has been validated through both simulation and FPGA-based implementation. The results obtained from hardware execution closely match those from simulation, confirming the correctness of the design.

The implemented division module achieves a maximum operating frequency of 40 MHz.

Operation	Expected	Obtained	Status
10.0/2.0	0x40a00000	0x40a00000	EXACT
1.0/4.0	0x3e800000	0x3e800000	EXACT
12.0/0.125	0x42c00000	0x42c00000	EXACT
3.0/2.0	0x3fc00000	0x3fc00000	EXACT
0.5/2.0	0x3e800000	0x3e800000	EXACT
1.0/1.0	0x3f800000	0x3f800000	EXACT
1.0/−2.0	0xbf000000	0xbf000000	EXACT

Table 5.1 – Verification results for exact division cases

Operation	Expected	Obtained	Status
1.0/0.0	0x7f800000	0x7f800000	CORRECT
-1.0/0.0	0xff800000	0xff800000	CORRECT
0.0/0.0	0x7fc00000	0x7fc00000	CORRECT
Inf/Inf	0x7fc00000	0x7fc00000	CORRECT
5.0/Inf	0x00000000	0x00000000	CORRECT
-5.0/Inf	0x80000000	0x80000000	CORRECT
NaN/5.0	0x7fc00000	0x7fc00000	CORRECT
5.0/NaN	0x7fc00000	0x7fc00000	CORRECT

Table 5.2 – Verification results for IEEE-754 special cases

Operation	Expected	Obtained	Error
-6.0/3.0	0xc0000000	0xbfffffff	1 ULP (0.00000596%)
7.0/3.0	0x40155555	0x40155554	1 ULP (0.00000596%)
1000.0/25.0	0x42200000	0x421fffff	1 ULP (0.00000298%)
-15.0/-3.0	0x40a00000	0x409fffff	1 ULP (0.00002384%)
π/π	0x3f800000	0x3f7ffffc	2 ULP (0.00000477%)
π/e	0x3f940ae9	0x3f93eedc	~1 ULP (0.00001157%)
1.5/4.5	0x3eaaaaab	0x3eaaaaaa	1 ULP (0.00000596%)
5.0/3.0	0x3fd55555	0x3fd55554	1 ULP (0.00000596%)

Table 5.3 – Verification results for high-precision floating-point division

The results demonstrate that the implemented division unit achieves high numerical accuracy, with errors typically within 1 ULP and a maximum deviation of 2 ULP. This level of precision is consistent with the expected behavior of IEEE-754 single-precision floating-point arithmetic and confirms the effectiveness of the Newton-Raphson approximation method. The small relative errors indicate that the design provides near-optimal accuracy while maintaining efficient hardware implementation.

5.2 Iterative Restoring Floating-Point Divider

5.2.1 Algorithm and Control Design

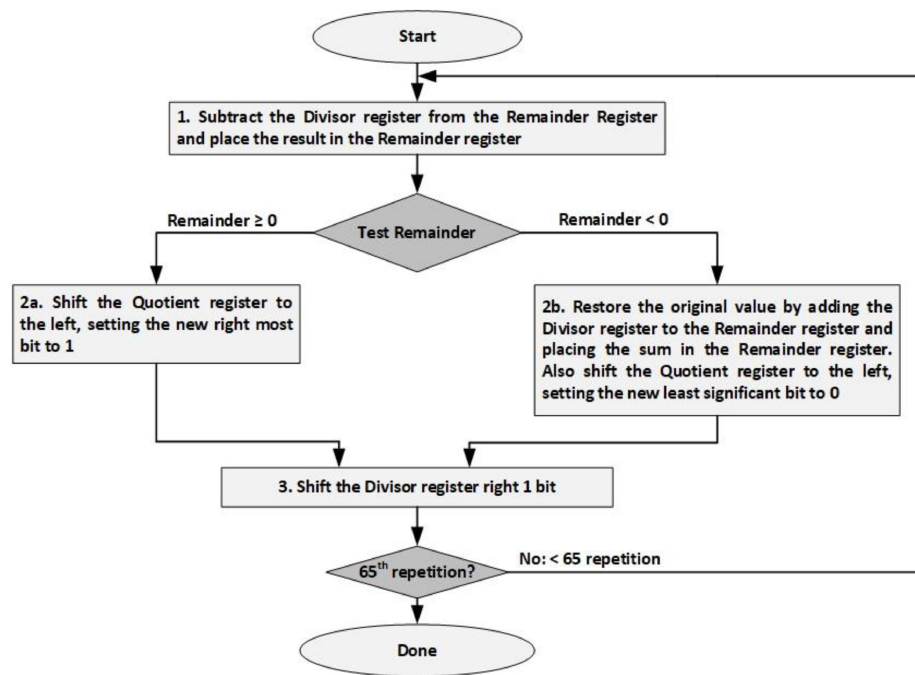


Figure 5.2 – Iterative Restoring Division Algorithm
[26]

5.2.2 Pipeline Stages of the Iterative Division Unit

The division operation is implemented using an iterative restoring division algorithm [26], organized into a three-stage pipeline. The second stage performs the division iteratively using a multi-cycle approach with loop unrolling. Figure 5.2 describes the general algorithm, where 65(= n) is the number of iterations which may vary.

- **Stage 1: Operand Extraction and Preprocessing**

The input operands A and B are decomposed into sign, exponent, and mantissa components. The implicit leading 1 is appended for normalized numbers, while denormalized values are handled separately.

Special cases are detected, including:

- NaN propagation
- Division by zero

- Infinity handling
- Zero operands

The result sign is computed as:

$$\text{sign}_{\text{result}} = \text{sign}_A \oplus \text{sign}_B$$

The exponent is initialized as:

$$\text{exp}_{\text{result}} = \text{exp}_A - \text{exp}_B + 127$$

To ensure correct division range, the mantissas are conditionally aligned such that:

$$1 \leq \frac{A}{B} < 2$$

Latency: 1 cycle

- **Stage 2: Iterative Division (Multi-Cycle Execution)**

This stage performs division using a restoring division algorithm with loop unrolling of two bits per cycle. [22]

At each iteration:

- The partial remainder is compared with the divisor
- Conditional subtraction is performed
- The quotient bits are generated
- The remainder is shifted left for the next iteration

Two quotient bits are computed per cycle:

$$\text{bit}_1, \text{bit}_2 \rightarrow \text{quotient update}$$

The iteration continues until all mantissa bits are processed (approximately 15 cycles).

A sticky bit is generated to capture remaining fractional information for rounding:

$$\text{sticky} = (\text{remainder} \neq 0)$$

Special cases bypass the iterative loop and directly produce results.

Latency: ~15 cycles (multi-cycle stage)

Throughput Impact: Pipeline stalls during execution

Finite State Machine for Stage 2

The iterative division unit is controlled by a finite state machine (FSM) that manages the multi-cycle execution of the division operation. The FSM transitions through several states to perform operand loading, iterative computation, and result generation.

- **IDLE**

The divider remains inactive in this state until a valid input is received. When the `valid_in` signal is asserted, the FSM transitions to the LOAD state.

- **LOAD**

In this state, the input operands and associated control signals are loaded into internal registers. The remainder register is initialized, and the iteration counter is reset. If a special case is detected (e.g., NaN, infinity), the FSM bypasses the iterative process and directly transitions to the DONE state. Otherwise, it proceeds to the ITERATE state.

- **ITERATE**

This state performs the core iterative division using a restoring division algorithm. The division loop is partially unrolled to compute two quotient bits per cycle. Each iteration consists of:

- * Comparing the current remainder with the divisor
- * Performing conditional subtraction
- * Updating the quotient register
- * Shifting the remainder for the next iteration

The iteration continues until all mantissa bits are processed. The iteration counter tracks progress and determines when the computation is complete.

- **FINALIZE**

Once the required number of iterations is completed, the final quotient is computed. The sticky bit is generated based on the remaining partial remainder, and the FSM prepares the output signals for the next stage.

- **DONE**

In this state, the `valid_out` signal is asserted to indicate that the result is ready. The FSM then returns to the IDLE state, awaiting the next division operation.

- **Stage 3: Normalization and Rounding**

The computed quotient is normalized to IEEE-754 format. Depending on the exponent range:

- Normal results are rounded using guard, round, and sticky bits
- Subnormal results are right-shifted appropriately
- Overflow results are set to infinity

Rounding is performed based on the selected rounding mode:

- Round to nearest (default)
- Toward zero
- Toward $\pm\infty$

The final result is packed as:

{sign, exponent[7 : 0], mantissa[22 : 0]}

IEEE-754 exception flags (inexact, overflow, underflow, invalid) are also generated.

Latency: 1 cycle

Total Latency: ~ 17 cycles

Operating Frequency: 55 MHz

Design Trade-off: Higher latency but improved global clock frequency

5.2.3 Verification Results

The simulation results confirm that the implemented floating-point unit satisfies the IEEE-754 accuracy requirements. All arithmetic operations produce results within an error bound of ± 2 ULP.

Register	Test Case	Expected Result	Obtained Result	ULP Error
r6	Division (π/e)	1.155727	1.155727	0
r9	Division by zero	0x7f800000 (Infinity)	0x7f800000	0
r18	π/π	1.000000	1.000000	0
r21	e/π	0.865256	0.865256	0
r23	NaN / π	NaN	NaN (propagated)	0
r24	$1/\pi$	0.318310	0.318310	0

Table 5.4 – FPGA verification results for floating-point operations

5.3 Final Selection of the Division Architecture

A comparison was performed between two floating-point division architectures: a Newton–Raphson (NR) based division module and an iterative division module. The evaluation considers operating frequency, numerical accuracy measured in Units in the Last Place (ULP), pipeline stalls, and hardware resource usage.

5.3.1 Simulation Results of the Entire Design Using Restoring Iterative Division

Register	Operation	Result	Expected	ULP Error
r3	add	5.859870	5.859870	0
r4	sub	0.423310	0.423310	1
r5	mul	8.539721	8.539721	0
r6	div	1.155727	1.155727	0

Table 5.5 – Verification of basic floating-point operations

Register	Test Case	Result
r9	Division by zero	7f800000 (Infinity)
r12	Very large $\times$ very small	1.000000
r13	Large + Large	2.0×10^{30}

Table 5.6 – Verification of IEEE-754 special cases

Register	Test Case	Result	ULP Error
r16	$\pi \times 1.0$	3.141590	0
r17	$\text{NaN} + \pi$	NaN (propagated)	0
r18	π / π	1.000000	0
r19	$0 - 0$	0.000000	0
r20	$1.0 + \infty$	∞	0
r21	e / π	0.865256	0
r22	$\text{NaN} \times \pi$	NaN (propagated)	0
r23	NaN / π	NaN (propagated)	0
r24	$1 / \pi$	0.318310	0
r25	$1 + \pi$	4.141590	0

Table 5.7 – Additional floating-point verification tests

Metric	Value
Total tests performed	10
Maximum ULP error	1
Average ULP error	0.10

Table 5.8 – ULP error analysis of the implemented floating-point unit

5.3.2 Simulation Results of The Entire Design Using NR Division

All arithmetic modules (addition, subtraction, multiplication, and division) were validated using simulation-based tests with an error bound of ± 2 ULP.

Register	Operation	Result	Expected	ULP Error
r3	add	5.859870	5.859870	0
r4	sub	0.423310	0.423310	1
r5	mul	8.539721	8.539721	0
r6	div	1.155727	1.155727	0

Table 5.9 – Verification of basic floating-point operations

Register	Test Case	Result
r9	Division by zero	7f800000 (Infinity)
r12	Very large $\times$ very small	1.000000
r13	Large + Large	2.0×10^{30}

Table 5.10 – Verification of IEEE-754 special cases

Register	Test Case	Result	ULP Error
r16	$\pi \times 1.0$	3.141590	0
r17	$\text{NaN} + \pi$	NaN (propagated)	0
r18	π / π	1.000000	1
r19	$0 - 0$	0.000000	0
r20	$1.0 + \infty$	∞	0
r21	e / π	0.865256	1
r22	$\text{NaN} \times \pi$	NaN	0
r23	NaN / π	NaN (propagated)	0
r24	$1 / \pi$	0.318310	0
r25	$1 + \pi$	4.141590	0

Table 5.11 – Additional floating-point verification tests

Metric	Value
Total tests performed	10
Maximum ULP error	1
Average ULP error	0.30

Table 5.12 – ULP error analysis of the implemented floating-point unit

5.3.3 Memory Utilization

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IOB (200)	BUFCTRL (32)	MMCM2_ADV (4)	BUFHCE (72)	BSCAT (4)
Top_Fpga	10606	15465	903	276	5074	8751	1855	117.5	10	21	3	1	1	
dbg_hub (dbg_hub)	448	741	0	0	221	424	24	0	0	0	1	0	0	
u_core (FPPipelinedProcessor)	10157	14711	903	276	4861	8326	1831	117.5	10	0	0	0	0	
execute (Execute)	3082	869	317	131	992	3082	0	0	10	0	0	0	0	
fp_adder (ieee754_adder)	905	141	15	0	274	905	0	0	0	0	0	0	0	
fp_divider (ieee754_div)	1004	266	287	131	351	1004	0	0	8	0	0	0	0	
lut_inst (lookup_table)	668	0	287	131	182	668	0	0	0	0	0	0	0	
nr1_inst (NR1)	63	0	0	0	40	63	0	0	4	0	0	0	0	
fp_multiplier (ieee754mult)	271	182	0	0	119	271	0	0	2	0	0	0	0	
fp_subtractor (ieee754_su)	906	141	15	0	262	906	0	0	0	0	0	0	0	
ila_inst (ila_0)	6304	12454	330	145	3575	4473	1831	117.5	0	0	0	0	0	

Figure 5.3 – NR Division Architecture Memory Utilisation

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Block RAM Tile (140)	DSPs (220)	Bonded IOB (200)	BUFCTRL (32)
Top_Fpga	10080	15306	616	145	5029	8225	1855	117.5	2	21	
dbg_hub (dbg_hub)	448	741	0	0	234	424	24	0	0	0	
u_core (FPPipelinedProcessor)	9631	14552	616	145	4804	7800	1831	117.5	2	0	
execute (Execute)	2554	707	30	0	824	2554	0	0	2	0	
fp_adder (ieee754_adder)	397	74	15	0	131	397	0	0	0	0	
fp_divider (ieee754_div)	487	241	0	0	180	487	0	0	0	0	
fp_multiplier (ieee754mult)	1266	176	0	0	404	1266	0	0	2	0	
fp_subtractor (ieee754_su)	408	78	15	0	143	408	0	0	0	0	
ila_inst (ila_0)	6307	12454	330	145	3599	4476	1831	117.5	0	0	

Figure 5.4 – Restoring Iterative Division Architecture Memory Utilisation

Memory Utilization of both the architectures show Newton Raphson methods incur higher resource utilization due to its multiplication based iterative structure , which increases LUT and DSP usage compared to the restoring division architecture.

5.3.4 Comparison

The comparison reveals a trade-off between latency, operating frequency, and hardware resource utilization. The Newton–Raphson divider achieves lower latency due to its parallel arithmetic operations but requires higher LUT utilization and operates at a lower clock frequency. In contrast, the iterative divider occupies less hardware and achieves a higher operating frequency, at the cost of increased stall cycles.

Metric	Newton–Raphson Divider	Iterative Divider
Clock Frequency	40 MHz	55 MHz (70 MHz without loop unrolling)
ULP Error	0.30	0.10
Pipeline Stalls	7 cycles	15 cycles (30 without loop unrolling)
Hardware Area	Larger utilization	Smaller utilization

Table 5.13 – Comparison between Newton–Raphson and Iterative Division Architectures

The Newton-Raphson (NR) division architecture offers several advantages over the iterative division module, including faster convergence, lower latency, and higher throughput. Unlike iterative division, which computes the quotient bit-by-bit, the NR method refines the reciprocal approximation exponentially, enabling the division to be completed in significantly fewer iterations. Furthermore, the use of fused multiply-add (FMA) units can further improve efficiency by reducing the number of intermediate operations and minimizing rounding errors.

In isolation, the NR-based approach represents a more efficient and high-performance solution. However, when considering the complete floating-point unit (FPU) architecture and the overall instruction set requirements, a broader design trade-off must be made.

The restoring division architecture emerges as a more suitable choice in this context. Division operations occur relatively infrequently in typical workloads, making higher latency more acceptable. Although the iterative approach introduces additional stall cycles, it enables a higher operating frequency due to its shorter critical path and simpler datapath. Additionally, it requires fewer hardware resources compared to the NR-based design.

Therefore, the restoring iterative divider provides a favorable trade-off between performance, hardware complexity, and overall system efficiency, making it well-suited for integration within the proposed processor architecture.

The Newton-Raphson method presents strong potential for future optimization through the integration of fused multiply-add instructions, which can enhance both computational efficiency and numerical accuracy.

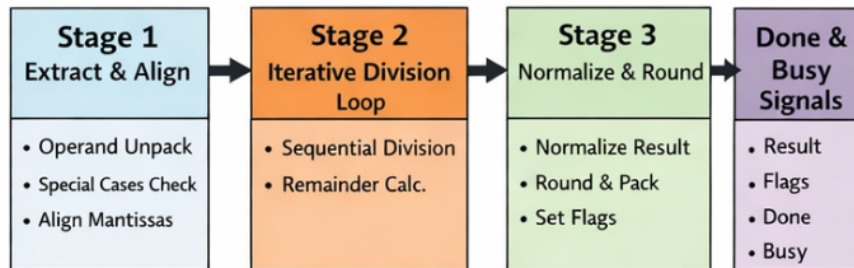


Figure 5.5 – Final Division Architecture

Chapter 6

Final Design Implementation on FPGA

6.1 Intellectual Property

An intellectual property core (IP core) is a functional block of logic or data used to make a field-programmable gate array (FPGA) or application-specific integrated circuit for a product.[27]

Complex system designs can be created by the Xilinx® Vivado® Design Suite IP integrator by instantiating and interconnecting IP cores from the Vivado IP catalog onto a design canvas.

This can be done interactively through the IP integrator design canvas GUI, or programmatically using a Tcl programming interface. Designs are typically constructed at the AXI-interface level for greater productivity but also can be manipulated at port level for more precise design control. [28]

As mentioned in Section 3.2.2 and Section 3.3 ILA and Clocking Wizards have been used.

6.1.1 Clocking Wizard

The Clocking Wizard is a LogiCORE™ IP provided with AMD Vivado and ISE software that simplifies the configuration of FPGA clocking resources.

It automatically generates HDL code to create and configure clock networks, including buffering, feedback, and timing parameters, while also allowing manual customization. Additionally, it produces timing analysis reports to ensure the designed clock system meets performance requirements. [28]

In this thesis, the Clocking Wizard IP is used to generate clock signals that meet the design specifications. It is instantiated in the top module, enabling proper configuration and distribution of the system clock.

```

1 clk_wiz_0 u_clkgen (
2     .clk_in1  (clk),
3     .reset    (reset),
4     .clk_out   (clk_core),
5     .locked   (mmcm_locked)
6 );

```

Listing 6.1 – Clock Wizard Instantiation

6.1.2 Integrated Logic Analyzer

The customization Integrated Logic Analyzer (ILA) IP core is a logic analyzer core that is used to monitor the internal signals of a design. The ILA core includes many advanced features of modern logic analyzers, including Boolean trigger equations, and edge transition triggers. Because the ILA core is synchronous to the design being monitored, all design clock constraints that are applied to your design are also applied to the components inside the ILA core. [28]

The Integrated Logic Analyzer (ILA) is utilized in this thesis to monitor internal signals on the FPGA, particularly for debugging register contents and verifying the correct operation of arithmetic units. It is instantiated as shown in Listing 6.2 to enable efficient and accurate debugging of the design.

```

1 ila_0 ila_inst (
2     .clk(clk),
3
4     // Existing probes for FPU results from Execute stage
5     .probe0(debug_add_result),
6     .probe1(debug_sub_result),
7     .probe2(debug_mul_result),
8     .probe3(debug_div_result),
9     .probe4(debug_monitor_pc),
10    .
11    .
12    .
13    .
14    .probe25(debug_reg_file[12]),
15    .probe26(debug_opcode),
16    .probe27(debug_decode_instruction_out)
17 );

```

Listing 6.2 – ILA Instantiation for Debugging

6.2 Design Verification

An XDC constraint file is provided to specify the physical pin assignments, clock definitions, and relevant timing constraints for the design.

The register file is initialized with a set of representative floating-point values, including standard constants such as π and e , as well as edge cases like zero, very large and very small numbers, infinity, and NaN. These values are used to verify the correctness and robustness of the floating-point unit under different operating conditions.

```

1 reg_file[1]  <= 32'h40490fd0; // f1  = 3.14159 (Pi)
2 reg_file[2]  <= 32'h402df84d; // f2  = 2.71828 (e)
3 reg_file[7]  <= 32'h3F800000; // f7  = 1.0
4 reg_file[8]  <= 32'h00000000; // f8  = 0.0
5 reg_file[10] <= 32'h7149f2ca; // f10 = 1.0e30 (very large)
6 reg_file[11] <= 32'h0da24260; // f11 = 1.0e-30 (very small)
7 reg_file[14] <= 32'h7F800000; // f14 = +Infinity
8 reg_file[15] <= 32'h7FC00000; // f15 = NaN (quiet NaN)

```

Listing 6.3 – Initialization of floating-point register values

An `imem.mem` file was defined to provide the RISC-V instruction sequence for single-precision floating-point operations. These instructions are used to verify the functionality of the floating-point unit (FPU). The sequence includes arithmetic operations such as addition, subtraction, multiplication, and division, executed across different combinations of registers. In addition, selected instructions are repeated to ensure consistent behavior and to evaluate pipeline stability in consecutive operations.

```

1 00310053 // FADD.S f0, f2, f3 funct7 = 0x00
2 083100D3 // FSUB.S f1, f2, f3 funct7 = 0x04
3 10310253 // FMUL.S f4, f2, f3 funct7 = 0x08
4 183102D3 // FDIV.S f5, f2, f3 funct7 = 0x0C
5 18C18253 // FDIV.S f5, f3, f12 funct7 = 0x0C
6 10310253 // FMUL.S f4, f2, f3
7 083100D3 // FSUB.S f1, f2, f3
8 00310053 // FADD.S f0, f2, f3
9 00240053 // FADD.S f0, f2, f8
10 00310053 // FADD.S f0, f2, f3
11 08310053 // FSUB.S f0, f2, f3
12 10310053 // FMUL.S f0, f2, f3
13 18310053 // FDIV.S f0, f2, f3

```

Listing 6.4 – Encoded floating-point instructions for FPU verification

Internal signals were probed and monitored using the Integrated Logic Analyzer (ILA). The observed register outputs in Figure 6.1 confirmed the correct operation of the design, with all results falling within the acceptable ULP error range.

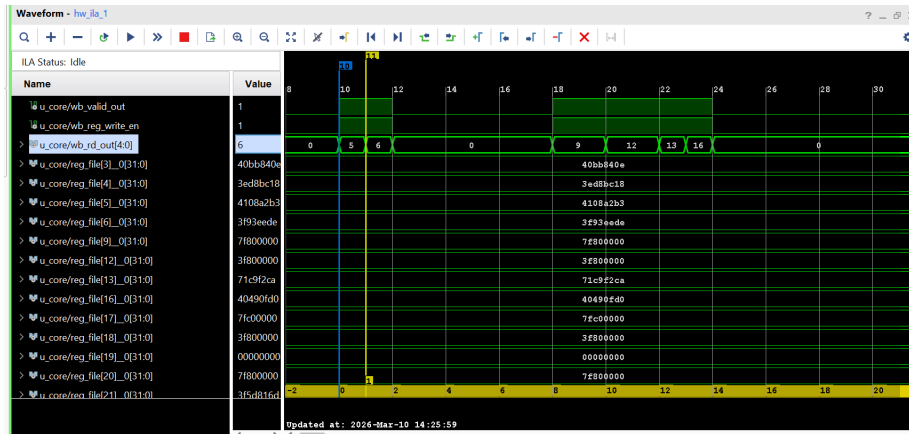


Figure 6.1 – Implementation Results using ILA

The floating-point unit developed in Part I has been integrated into the NOVACore processor architecture, as described in Part III.

Following the integration of the iterative division module into the existing design as discussed in Section 5.3, the complete system was thoroughly verified through both simulation and FPGA-based implementation.

6.3 Final Architecture

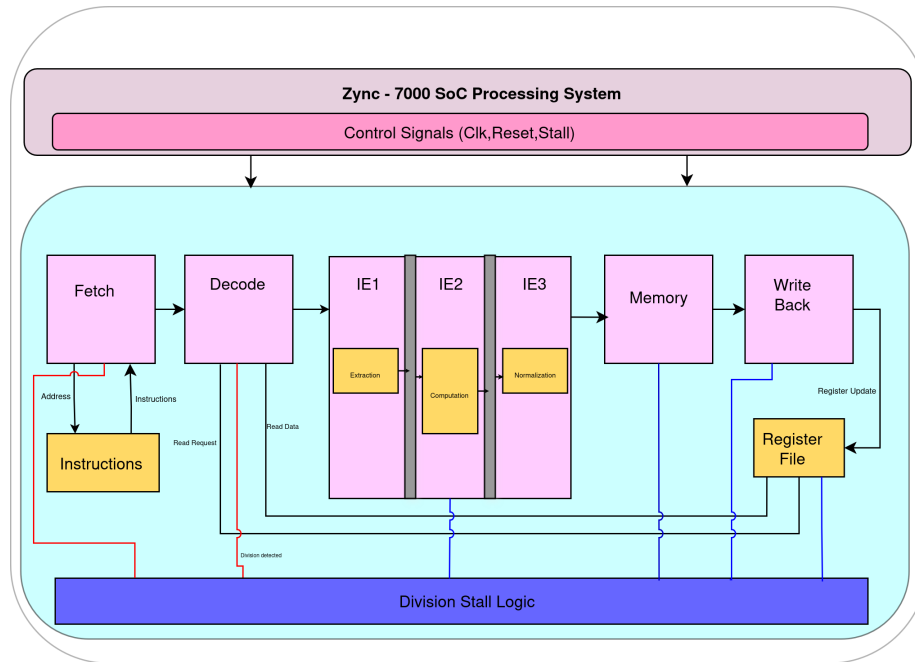


Figure 6.2 – Final FP Arithmetic Unit Architecture

As described in Figure 6.2, a fully pipelined floating-point arithmetic unit (FPU) has been designed.

The proposed design is integrated with a forwarding unit, hazard detection logic, memory operations, and support for the floating-point instruction set architecture (ISA). Additionally, a division stall mechanism is incorporated to ensure correct pipeline execution when division is detected. These components and their interactions are described in detail in Part III.

Part II

**Development of the Customized
RISC-V F extension to support
floating-point instructions and
arithmetic operations.**

Chapter 7

RISC V Floating Point ISA.

7.1 Overview

RISC-V has become an open-source instruction set architecture (ISA) for the revolution that is providing flexibility and customization options much wider than those associated with proprietary ISAs like x86 and ARM. With an open standard, RISC-V allows researchers and hardware developers to customize the ISA according to particular application needs, thus making it the perfect fit for a variety of computing applications. [1] [8]

Category	Instructions
Arithmetic	FADD.S, FSUB.S, FMUL.S, FDIVS, FSQRT.S
Fused Multiply-Add	FMADD.S, FMSUB.S, FNMSUB.S, FNMADD.S
Comparison	FEQ.S, FLT.S, FLE.S
Min/Max	FMIN.S, FMAX.S
Sign Manipulation	FSGNJ.S, FSGNJN.S, FSGNJX.S
Conversion	FCVT.W.S, FCVT.WU.S, FCVT.S.W, FCVT.S.WU
Move	FMVX.W, FMVW.X
Classification	FCLASS.S
Memory	FLW, FSW

Table 7.1 – Single-precision floating-point instructions in the RISC-V F-extension

Most floating-point arithmetic instructions in the RISC-V F extension use an R-type instruction format and are encoded with the OP-FP opcode (0x53). However, floating-point memory operations such as FLW and FSW follow the standard I-type and S-type formats, respectively. Additionally, fused multiply-add instructions use a specialized four-operand instruction format (R4-type).

7.2 Customized Floating-Point ISA

The hardware implementation prioritizes the core arithmetic operations required for scientific computing and signal processing.

funct7	rs2	rs1	rm	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Table 7.2 – R-Type floating-point instruction format in the RISC-V F-extension

For example, the `FADD.S` instruction performs single-precision floating-point addition using two source registers (`rs1` and `rs2`) and writes the result to the destination register `rd`.

R Type Instructions

The following instructions are supported by the architecture.

- **Arithmetic Operations:**

- `FADD.S`
- `FSUB.S`
- `FMUL.S`
- `FDIV.S`

- **Conversion Instructions:**

- `FCVT.W.S`, `FCVT.WU.S`
- `FCVT.S.W`, `FCVT.S.WU`

- **Comparison Instructions:**

- `FLE.S`, `FEQ.S`, `FLT.S`

- **Minimum/Maximum Instructions:**

- `FMAX.S`, `FMIN.S`

- **Move Instructions:**

- `FMV.W.X`
- `FMV.X.W`

Memory Operations

The following floating-point memory access instructions are supported:

- `FLW` (Floating-Point Load Word) - I Type
- `FSW` (Floating-Point Store Word) - S Type

7.3 Rationale for the Selected Instruction Subset

As mentioned in Section 7.1, the RISC-V architecture provides the flexibility to design and extend the instruction set according to application-specific requirements.

The primary objective of this thesis was the development of a functional floating-point unit integrated into a RISC-V processor. Consequently, only the subset of floating-point instructions necessary to support the implemented arithmetic operations and system functionality was selected.

Instruction Category	Instructions Not Implemented
Fused Multiply-Add Operations	FMADD.S, FMSUB.S, FNMSUB.S, FNMADD.S
Square Root Operation	FSQRT.S
Sign Manipulation Instructions	FSGNJ.S, FSGNJN.S, FSGNJX.S
Classification Instruction	FCLASS.S
Dynamic Rounding Mode Support	DYN rounding mode using FCSR

Table 7.3 – Floating-point instructions from the RISC-V F-extension not implemented in the proposed architecture

The instructions listed in Table 7.3 were not implemented in the current design. These instructions were not required for the functional evaluation of the floating-point unit. The focus of this work was the implementation and evaluation of the core floating-point arithmetic operations and their integration within the RISC-V processor pipeline.

Another method used to determine the required instruction subset was the analysis of programs compiled using the RISC-V GNU toolchain. Test programs written in C were compiled and converted into assembly and binary representations using the toolchain. By examining the generated assembly code, it was possible to identify the floating-point instructions that were actually required for program execution. This approach helped ensure that the implemented instruction subset was sufficient to support the intended floating-point operations of the processor. This will be discussed in details in Chapter 8.

Chapter 8

RISC-V Toolchain

8.1 Overview

RISC-V is an open source modular and scalable emerging instruction set. As the RISC-V architecture gradually matures in the field of contemporary chips, the RISC-V software ecosystem is also gradually prospering. Some mainstream tool chains and operating systems have supported RISC-V architecture since the beginning, and now gradually support multiple RISC-V expansion directive. Although there are many works dedicated to advancing RISC-V instruction extensions to adapt to various scenarios under different computing power requirements, and exploring the RISC-V software ecosystem, there is no work on existing tool chains and operating systems to extend RISC-V Conduct systematic research and conclusions on the support of the instructions.[29]

8.2 Compiler

Compiler is a software tool that converts high-level language into machine code. It plays a major role in software development since every software must be processed by a compiler [30]

The compiler is an important part of the RISC-V software ecosystem, which mainly completes processes such as preprocessing, compilation, assembly, and linking. Currently, the compilers that support RISC-V include GNU Compiler Collection (GCC), LLVM and TVM, etc. [29]

GCC

The GCC is a classical compiler that provides front ends for C, C++, and Objective-C languages, etc. It supports many different ISA (e.g., x86, ARM, RISC-V) in its

back-end and is the first available compiler for RISC-V. GCC began to add support for RISC-V in version 7.1. GCC 12 released in 2022 supported all the instruction sets defined in the latest ISA specification (version 20191213) released by RISC-V International Foundation.[29]

8.3 RISC V Modularity

An important development goal of the open ISA was to facilitate the development of fast, small, high-performance and energy-efficient processors. Implementation is possible both on FPGAs and ASICs. The three available integer-based ISAs are primarily distinguished by bit width. As the name suggests, the architect versions RV32I, RV64I and RV128I have a width of 32, 64 and 128 bit respectively. In addition, RV32E also exists with only 16 registers, a version specially designed for embedded systems. [31]

Ext.	Name	Description
M	Multiply	Commands for integer multiplication and division.
A	Atomic	Protected memory access for multi-CPU synchronization.
F	Floating-Point	Single-precision (32-bit) IEEE 754-2008 operations.
D	Double-Precision	Double-precision (64-bit) IEEE 754-2008 operations.
Q	Quad-Precision	Quad-precision (128-bit) IEEE 754-2008 operations.
C	Compressed	16-bit instruction encoding for code size reduction.

Table 8.1 – Standard RISC-V Extensions

Modularity and expandability are the greatest strengths of this architecture, which makes it possible to design processors customized to the application. Thus the required commands are only added for the respective applications. This in turn leads to a core size with maximum efficiency as well as the ability to manufacture the cores in a cost-efficient manner.[31]

8.4 RISC-V Open Source Toolchain

The GCC project (GNU Compiler Collection) open source initiative facilitates compiler support for several RISC V ISA variants. GCC version 7.1 and higher supports RISC-V.

The software infrastructure for the NOVACore is built using the **riscv32-unknown-elf** cross-compiler. The build system utilizes a specific set of compiler and linker flags to ensure the generated machine code is compatible with the implemented RV32IMF architectural extensions.

The floating-point extension (F) is required for executing IEEE-754 single-precision floating point instructions such as `fadd.s`, `fsub.s`, `fmul.s`, `fdiv.s`, etc. which are supported by the implemented processor pipeline.

The test programs used to verify the floating-point processor were compiled using the `riscv32-unknown-elf-gcc` cross-compiler targeting a 32-bit RISC-V architecture.

The target architecture is `rv32imf_zicsr`, which includes the base integer instruction set (I), integer multiplication (M), single-precision floating-point operations (F), and control and status register instructions (Zicsr). The following compiler and linker flags were used in the build system.

Flag	Description
<code>-march=rv32imf_zicsr</code>	Targets the RV32I architecture with multiplication, floating-point support, and CSR instructions.
<code>-mabi=ilp32</code>	Specifies the ILP32 ABI where integers, longs, and pointers are 32-bit wide.
<code>-Os</code>	Optimizes the generated code for minimal size, which is beneficial for embedded systems.
<code>-ffreestanding</code>	Indicates that the program runs in a freestanding environment without an operating system.
<code>-fno-builtin</code>	Prevents the compiler from replacing functions with built-in implementations.
<code>-nostartfiles</code>	Avoids linking standard startup files, which is typical for bare-metal systems.
<code>-Wl,-gc-sections</code>	Removes unused sections during linking to reduce the final binary size.

Table 8.2 – Important compiler and linker flags

8.5 RISC V Simulator (SPIKE)

The Spike simulator is a command-line application for RISC-V development. It allows for both 32-bit and 64-bit simulations and supports multiple variants of RISC-V types. It's an alternative to using the Segger emSim simulator, which has both a graphical mode and a command line mode. Spike is typically used with a special interpreter called `pk`. It can also be used stand-alone. [32]

8.6 RISC-V Compilation and Toolchain Flow

The test application in C is compiled to an executable file ELF.

```
riscv64-unknown-elf-gcc -nostartfiles -Ttext 0 -march=rv32if
-mabi=ilp32 -o Test Test.c
```

The compiled RISC-V program is first converted from ELF format to a raw binary file using the `objcopy` utility. This removes all metadata and retains only the executable machine code required for hardware execution.

```
riscv64-unknown-elf-objcopy -I elf-riscv32 -O binary Test Test.bin
```

The generated binary file is inspected using the `od` utility, which displays the contents as 32-bit hexadecimal words:

```
od -t x4 Test.bin
```

Subsequently, the binary file is converted into a formatted hexadecimal representation suitable for FPGA memory initialization using the `hexdump` utility:

```
hexdump -v -e '"0x" /4 "%08x" ",\n"' Test > Test.hex
```

A dump file can also be generated which gives an overview of the Hex Memory as well as the assembly level code.

```
riscv64-unknown-elf-objdump -d Test > Test.dump
```

```

1 00000000 <main>:
2  0:  fe010113          add     sp,sp,-32
3  4:  00812e23          sw      s0,28(sp)
4  8:  02010413          add     s0,sp,32
5  c:  fec42707          flw     fa4,-20(s0)
6 10:  fe842787          flw     fa5,-24(s0)
7  .
8  .
9  .
10 .
11 34:  08f777d3          fsub.s  fa5,fa4,fa5
12 38:  fef42627          fsw     fa5,-20(s0)
13 3c:  fe442707          flw     fa4,-28(s0)
14 40:  fec42787          flw     fa5,-20(s0)
15 44:  18f777d3          fdiv.s  fa5,fa4,fa5
16 48:  fef42227          fsw     fa5,-28(s0)
17 .
18 .
19 .

```

Listing 8.1 – Dump File Overview

8.7 Functional Verification

The functional verification of the floating-point unit was carried out through a sequence of steps involving program development, compilation, and generation of executable files for the target processor.

- **Step 1: Test Program Development**

A test application was developed in the C programming language to evaluate the functionality of the floating-point operations supported by the processor.

- **Step 2: Compilation Using the RISC-V Toolchain**

The application was compiled using the appropriate RISC-V cross-compilation toolchain (`riscv32-unknown-elf-gcc`) targeting the implemented processor architecture.

- **Step 3: Build Configuration**

A Makefile was created to manage the build process and consolidate the required compiler and linker flags for generating the target executable.

- **Step 4: Generation of Output Files**

Executing the Makefile generates several output files required for the verification process, including assembly (`.s`), binary (`.bin`), and memory initialization (`.coe`) files.

8.8 Instruction-Level Representation of Test Programs

```
1 int test_fpu_logic_add() {
2     volatile int a = 20;
3     volatile int b = 6;
4
5     float f1 = (float)a; // fcvt.s.w
6     float f2 = (float)b; // fcvt.s.w
7
8     float sum = f1 + f2; // fadd.s
9
10    return (int)sum;      // fcvt.w.s
11 }
12
13 int main() {
14     volatile int result = test_fpu_logic_add();
15     return 0; }
```

Listing 8.2 – Floating-point addition test in C

As discussed in Section 8.7, a test program was developed in C to verify the functionality of the floating-point unit. The function shown above demonstrates a simple floating-point addition using the RISC-V FPU. Two integer values, 20 and 6, are first converted into single-precision floating-point representations. The addition is then performed in the floating-point domain, and the result is subsequently converted back to an integer. This program serves as a basic test case to validate the correct operation of the floating-point unit.

In the next step, the application is compiled using the RISC-V toolchain. This process translates the high-level C program into assembly-level instructions and generates the corresponding machine code. The resulting binary is further processed to produce memory initialization files (e.g., .coe), which are used to load the instructions into the processor's instruction memory.

8.8.1 Assembly-Level Implementation without the F Extension

The command `riscv64-unknown-elf-gcc -S -march=rv32i -mabi=ilp32` is used to generate assembly code from a C source file targeting a 32-bit RISC-V architecture without floating-point support.

```

1 test_fpu_logic_add:
2     addi    sp,sp,-48
3     sw      ra,44(sp)
4     sw      s0,40(sp)
5     addi    s0,sp,48
6     li      a5,20
7     sw      a5,-32(s0)
8     li      a5,6
9     sw      a5,-36(s0)
10    lw      a5,-32(s0)
11    mv      a0,a5
12    call     __floatsisf      # int -> float
13    mv      a5,a0
14    sw      a5,-20(s0)
15    lw      a5,-36(s0)
16    mv      a0,a5
17    call     __floatsisf      # int -> float
18    mv      a5,a0
19    sw      a5,-24(s0)
20    lw      a1,-24(s0)
21    lw      a0,-20(s0)
22    call     __addsf3         # software float addition
23    mv      a5,a0
24    sw      a5,-28(s0)
25    lw      a0,-28(s0)
26    call     __fixsfsi        # float -> int
27    mv      a5,a0
28    mv      a0,a5

```

```

29      lw      ra,44(sp)
30      lw      s0,40(sp)
31      addi    sp,sp,48
32      jr      ra
33      .size   test_fpu_logic_add, .-test_fpu_logic_add

```

Listing 8.3 – RISC-V assembly generated for floating-point addition without hardware FPU support

The assembly code shown in Listing 8.3 illustrates the execution of floating-point operations in the absence of hardware FPU support. Instead of using dedicated floating-point instructions, the compiler generates calls to software routines provided by the compiler runtime library.

The instructions `__floatsisf` are used to convert integer values into single-precision floating-point format. The floating-point addition is performed by the function `__addsf3`, which implements the operation using integer arithmetic. Finally, the result is converted back to an integer using `__fixsfsi`.

This demonstrates that, without the F extension, floating-point operations are handled through software emulation, resulting in increased instruction overhead compared to hardware-based execution.

8.8.2 Assembly Level Code with F extension

The command `riscv64-unknown-elf-gcc -S -march=rv32if -mabi=ilp32` is used to generate assembly code from a C source file targeting a 32-bit RISC-V architecture with floating-point support.

```

1 test_fpu_logic_add:
2     addi    sp,sp,-48
3     sw      s0,44(sp)
4     addi    s0,sp,48
5     li      a5,20
6     sw      a5,-32(s0)
7     li      a5,6
8     sw      a5,-36(s0)
9     lw      a5,-32(s0)
10    fcvt.s.w    fa5,a5
11    fsw      fa5,-20(s0)
12    lw      a5,-36(s0)
13    fcvt.s.w    fa5,a5
14    fsw      fa5,-24(s0)
15    flw      fa4,-20(s0)
16    flw      fa5,-24(s0)
17    fadd.s    fa5,fa4,fa5
18    fsw      fa5,-28(s0)
19    flw      fa5,-28(s0)

```

```

20      fcvt.w.s a5,fa5,rtz
21      mv      a0,a5
22      lw      s0,44(sp)
23      addi    sp,sp,48
24      jr      ra
25      .size   test_fpu_logic_add, .-test_fpu_logic_add
26      .align  2
27      .globl  main
28      .type   main, @function

```

Listing 8.4 – RISC-V assembly corresponding to floating-point addition

The generated assembly code shows the sequence of instructions executed by the processor. Integer values are first loaded from memory and converted into floating-point format using `fcvt.s.w`. The floating-point addition is performed using the `fadd.s` instruction. Finally, the result is converted back to an integer using `fcvt.w.s` and stored in memory.

Comparison

A comparison between Listing 8.4 and Listing 8.3 highlights the fundamental difference in the execution of floating-point operations. When the F extension is enabled, floating-point computations are performed directly by dedicated hardware instructions such as `fcvt.s.w`, `fadd.s`, and `fcvt.w.s`. This results in a compact sequence of instructions with minimal overhead.

In contrast, when the F extension is not available, floating-point operations are implemented using software routines provided by the compiler runtime library. Instead of single instructions, multiple function calls such as `__floatsisf`, `__addsf3`, and `__fixsfsi` are invoked to emulate floating-point behavior using integer arithmetic. This significantly increases the number of executed instructions and introduces additional function call overhead.

Therefore, the presence of the F extension enables efficient hardware-based execution of floating-point operations, while its absence leads to slower software-based emulation with higher computational cost.

Instruction Memory

```

1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  00000097
4  04408093
5  30509073
6  ...
7  00c12783

```

```
8 00000513
9 d00777d3
10 d007f753
11 00e7f7d3
12 c00797d3
13 00f12223
14 01010113
15 00008067
16 ;
```

Listing 8.5 – Memory initialization file (.coe) for instruction memory

The above memory initialization file contains the machine-level instructions in hexadecimal format. These instructions are generated from the compiled program and are used to initialize the instruction memory of the processor on the FPGA. Each entry corresponds to a 32-bit RISC-V instruction.

A comprehensive verification of the floating-point unit based on these instructions is presented in Part III.

Part III

Integration and Verification of the FPU within the NOVACore RISC-V Processor Architecture

Chapter 9

NovaCore In Order Processor Architecture

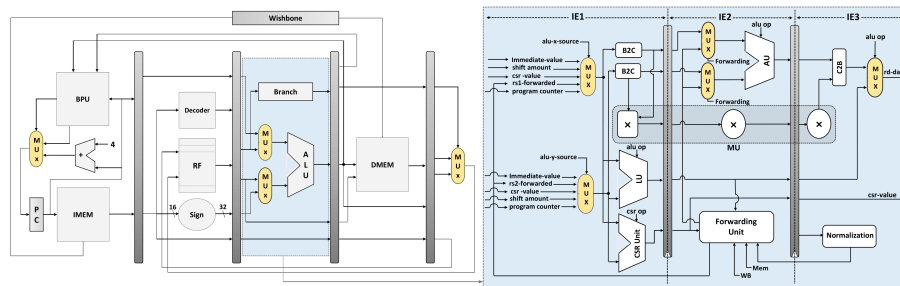


Figure 9.1 – NovaCore Architecture
[10]

9.1 The NOVACore

The NOVACore is a straightforward 7-stage in-order RISC-V processor implemented in VHDL for FPGA use. It supports the 32-bit RV32IZmmul-Zicsr ISA, providing a RISC-V processor with a base set of integer arithmetic instructions (RV32I), a multiplication extension excluding division (Zmmul), and control and status register manipulation instructions (Zicsr) as per Specification version 2.0. Additionally, it supports significant portions of the machine mode defined in the RISC-V Privileged Architecture Specification v1.10.

The processor has been tested on the Xilinx Zynq-7000 family (XC7Z020-1CLG400C) and Ultra96-V2 which is an Arm-based, AMD Xilinx Zynq UltraScale+™ MPSoC development board using the example SoC design.

Synthesis and implementation have been tested on various Xilinx's Vivado toolchain versions, most recently version 2023.1. [10]

9.2 Features

The NOVACore processor architecture incorporates several design features aimed at achieving efficient execution and flexible system integration. [10]

Processor Features

The implemented processor architecture is based on the following key features:

- 32-bit RV32I Zmmul Zicsr ISA, providing a RISC-V processor with a base set of integer arithmetic instructions (RV32I), a multiplication extension excluding division (Zmmul), and control and status register manipulation instructions (Zicsr, version 2.0).
- Support for large parts of the machine mode defined in the RISC-V Privileged Architecture (version 1.10).
- Support for up to 8 individually maskable external interrupts (IRQs).
- 7-stage RISC pipeline including a 3-step Ternary Encoded Instruction Execution stage (IE: IE1, IE2, IE3).
- 1-bit Dynamic Branch Prediction Unit (D-BPU).
- Optional instruction cache.
- Support for the Wishbone bus (version B4).

9.3 Peripherals

These peripherals provide essential functionality for communication, timing, and system control, enabling flexible and scalable SoC integration. [10]

Wishbone-Compatible Peripherals

The project includes a set of Wishbone-compatible peripherals designed for integration in system-on-chip (SoC) architectures based on the NOVACore processor. The main peripherals are summarized as follows: [10]

- **Timer:** A 32-bit timer module with compare interrupt capability.

- **GPIO:** A configurable-width general-purpose input/output (GPIO) module.
- **Memory:** A block RAM module used for on-chip data and instruction storage.
- **UART:** A universal asynchronous receiver-transmitter (UART) module featuring hardware FIFOs, configurable baud rate, and support for RX/TX interrupts.

Next Steps

Currently, the processor design supports only integer operations. To extend its functionality, a floating-point unit (FPU) has been integrated and tested. The successful integration and verification of the FPU are described in Chapter 10.

Chapter 10

Integration of the Floating-Point Unit into NOVACore

10.1 Overview of the Integration

Alongside the integer instruction set architecture (ISA), a subset of the floating-point ISA has been designed and integrated to support the functionality of the Floating-Point Unit (FPU). This subset has previously been introduced in Section 7.1. The subsequent sections detail the changes implemented in each stage of the RISC-V processor.

10.2 Floating-Point Register File

The floating-point register file is implemented as a separate entity from the integer register file to support the execution of floating-point instructions defined in the ISA. It consists of 32 registers, each 32 bits wide, corresponding to the single-precision IEEE-754 format.

The register file is designed with two read ports and one write port, allowing simultaneous access to two source operands and one destination operand. This enables efficient execution of floating-point operations within the pipeline.

Floating-point instructions exclusively access this register file for operand fetch and result write-back. However, certain instructions, such as `fmv.w.x` and `fcvt.s.w`, require data read from the integer register file and write to the floating-point register file. Similarly, instructions like `fmv.x.w`, `flt.s.feq.s`, `fle.s` and `fcvt.w.s` read data from the floating-point register file and write to the integer register file.

The control unit generates appropriate read and write enable signals to ensure correct data movement between the two register files. During the decode stage,

the instruction type determines whether the integer or floating-point register file is accessed.

This separation of register files allows parallel handling of integer and floating-point data paths while maintaining compatibility with the RISC-V floating-point ISA. Unlike the integer register file, in which `r0` is fixed to zero, the floating-point register file does not provide a dedicated zero register.

10.3 Decode Stage Modifications

To ensure correct identification of floating-point instructions, the corresponding opcodes for the Floating-Point Unit (FPU) have been incorporated into the decode stage.

Immediate Decoder

Although most floating-point instructions do not require immediate values, memory access operations such as `FLW` and `FSW` depend on integer addition and immediate offsets for effective address calculation. Therefore, the immediate decoder has been extended to properly support these instructions.

Control Unit

While most floating-point operations exclusively access the floating-point register file, certain instructions require interaction between integer and floating-point registers.

Instructions such as `FMV.W.X` and `FCVT.W.X` read data from the integer register file and write to the floating-point register file. Conversely, instructions like `FLT.S`, `FLE.S`, `FEQ.S`, `FMV.X.W`, and `FCVT.X.W` read from the floating-point register file and write results to the integer register file.

Accordingly, the control unit has been modified to generate appropriate control signals for correct read and write operations across both register files.

10.4 Execute Stage Modifications

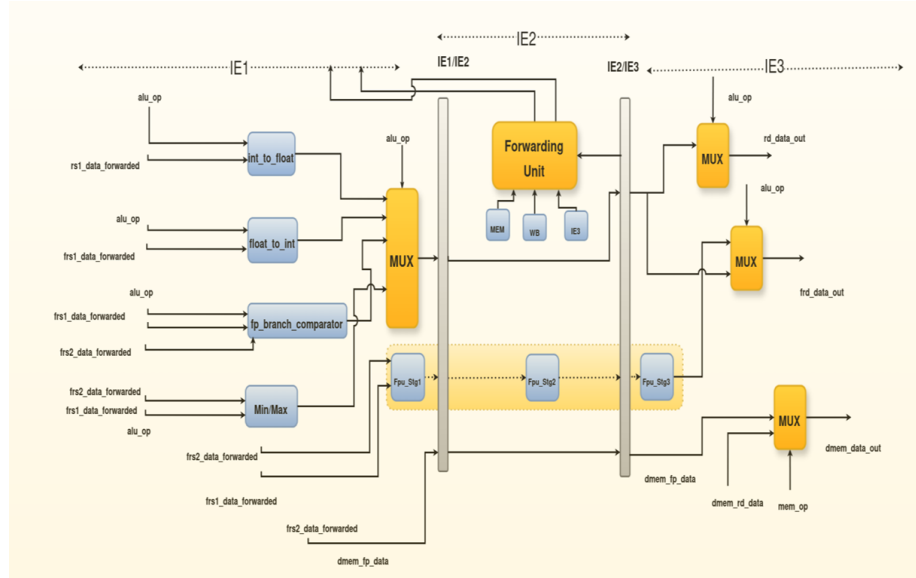


Figure 10.1 – Execution Architecture

The execution stage is partitioned into three pipeline stages. It is augmented with a forwarding unit and a hazard detection unit, which will be discussed in subsequent sections.

Floating-point arithmetic operations are implemented as multi-cycle operations spanning three pipeline stages, whereas the remaining floating-point instructions are executed within a single cycle.

IE1: First Execution Stage

The first stage of the floating-point arithmetic unit is integrated into this stage. The outputs generated in this stage are registered and forwarded to the next execution stage (IE2).

In addition to arithmetic operations, several floating-point instructions are executed within this stage, including:

- Floating-point to integer conversion (`fcvt.w.s`)
- Integer to floating-point conversion (`fcvt.s.w`)
- Floating-point comparison operations
- Floating-point result to integer register data movement operations (`fmv.x.w`)

- Floating-point Minimum Maximum Operations(`fmin.s`, `fmax.s`)

The results of these operations are written to the integer data path via a multiplexer, which selects the appropriate output based on the `alu_op` control signal.

IE2: Second Execution Stage

The second stage of the floating-point arithmetic unit, corresponding to the main computation phase, is integrated into this stage. The intermediate results generated in this stage are registered and forwarded to the next execution stage (IE3).

For floating-point arithmetic operations, this stage performs the core computation, such as addition, subtraction, multiplication, or division, depending on the instruction.

For all other floating-point instructions, the results computed in the previous stage are propagated through this stage without modification and forwarded to IE3.

IE3: Third Execution Stage

The third stage of the floating-point arithmetic unit, corresponding to the last normalization phase, is integrated into this stage. The final results of each arithmetic operation is muxed to the result signal based on `alu_op`.

The multiplexers are implemented in the final stage of the execution pipeline to route results to the appropriate destination based on the instruction type.

- **MUX 1: Integer Register Write-Back**

This multiplexer selects the appropriate result of floating-point operations that produce integer outputs and routes them to the integer register file. The supported operations include:

- `fmv.x.w`
- `fcvt.w.s`
- `fle.s`, `flt.s`, `feq.s`

- **MUX 2: Floating-Point Register Write-Back**

This multiplexer selects the result of floating-point operations that produce floating-point outputs and routes them to the floating-point register file. The supported operations include:

- `fmv.w.x`
- `fcvt.s.w`
- `fadd.s`, `fsub.s`, `fmul.s`, `fdiv.s`
- `fmin.s`, `fmax.s`

- **MUX 3: Memory Data Routing**

This multiplexer routes memory data for both integer and floating-point store operations. Depending on the memory type `sw` or `fsw` operations, it selects the appropriate data to be written to memory (`dmem_data_out`)

10.5 Memory and Writeback Stages Modifications

Appropriate modifications have been incorporated into the memory and writeback stages to support extended functionality.

10.6 Pipeline Control and Hazard Handling

10.6.1 Division Stall Logic

Unlike other floating-point arithmetic operations such as `fadd.s`, `fsub.s`, and `fmul.s`, the `fdiv.s` instruction requires approximately 17 clock cycles to complete. Due to this multi-cycle behavior, a dedicated counter based stall mechanism is introduced to ensure correct pipeline execution.

When a division instruction is detected in the decode stage, the fetch and decode stages are stalled until the execution stages become available. This prevents subsequent instructions from entering the pipeline while the division unit is being prepared.

Once the division operation enters the execution stage, the entire pipeline is stalled until the division operation is completed. The stall is released when the `div_done` signal is asserted, indicating that the result is ready for further propagation through the pipeline.

This stall mechanism ensures correct handling of multi-cycle division operations while maintaining pipeline consistency and preventing data hazards.

10.6.2 RAW Hazards

Since the processor follows an in-order execution model, only Read-After-Write (RAW) data hazards need to be considered.

RAW hazards occur when an instruction attempts to read a register whose value is yet to be written by a preceding instruction. This situation is particularly relevant in a pipelined architecture, where multiple instructions are executed concurrently.

To handle RAW hazards, a combination of forwarding and pipeline stalling mechanisms is employed. The forwarding unit resolves most data dependencies by providing operand values directly from later pipeline stages, thereby avoiding

unnecessary delays. In cases where forwarding is not sufficient, such as certain multi-cycle floating-point operations, pipeline stalls (or insertion of NOPs) are introduced to ensure correct execution.

10.6.3 Forwarding Unit

The forwarding unit enables data to be routed directly from later stages of the pipeline to earlier stages where it is required. In the proposed design, forwarding paths are established from the end of the IE3 stage, the memory (MEM) stage, and the write-back (WB) stage.

This allows dependent instructions to access the most recent operand values without waiting for them to be written back to the register file, thereby reducing pipeline stalls and improving overall performance.

10.6.4 Pipeline Stalling Mechanism

The pipeline stalling mechanism is employed to ensure correct execution in situations where data dependencies cannot be resolved through forwarding alone.

As discussed previously, the pipeline is stalled when a division instruction is detected, due to its multi-cycle execution latency. In this case, instruction fetch and decode are halted until the division operation is completed, preventing incorrect data propagation.

In addition to division-related stalls, hazards arising from load and memory operations are also handled using pipeline stalling. For example, in load-use scenarios where the required data is not yet available for forwarding, the pipeline is stalled until the data is retrieved from memory and becomes available for subsequent instructions.

This mechanism ensures correct program execution by preventing data hazards, while maintaining overall pipeline consistency.

10.7 Final Architecture

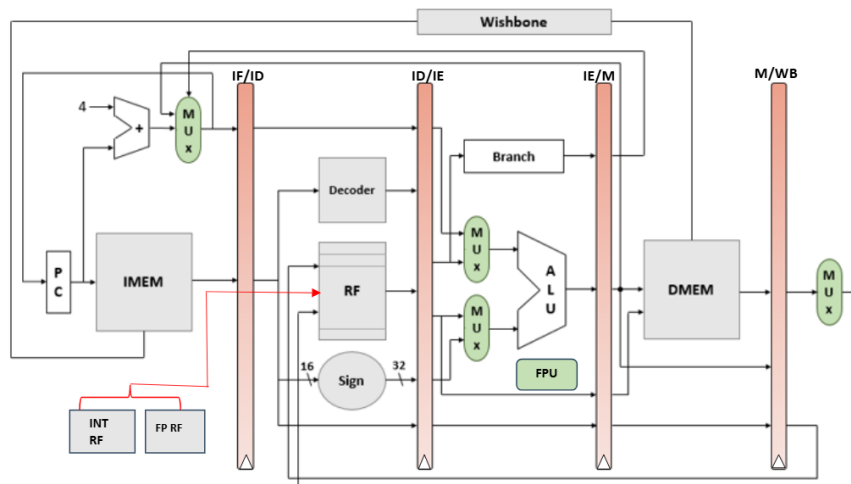


Figure 10.2 – Architecture of the NovaCore processor with integrated floating-point unit (FPU).

The final architecture integrates a seven-stage pipelined RISC-V processor with a dedicated floating-point unit (FPU) in the execution stage. The execution stage architecture is described in Figure 10.1. Separate integer and floating-point register files are employed, with appropriate data forwarding and memory interfaces to support both operation types. The design ensures efficient instruction flow across pipeline stages while enabling seamless execution of floating-point operations.

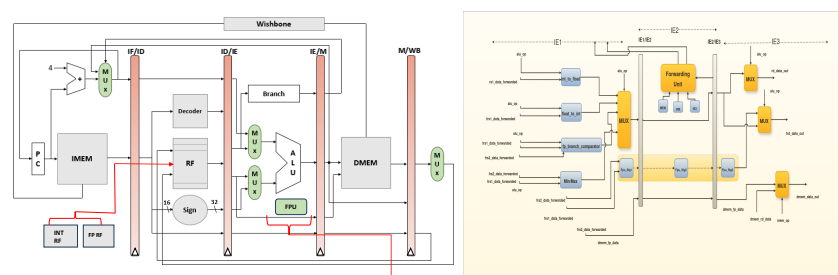


Figure 10.3 – Final NovaCore architecture.

Chapter 11

Results and Discussion

11.1 Simulation Based Verification

11.1.1 Verification I

After integrating the Floating-Point Unit (FPU), the complete design was verified using a variety of test applications. A representative test case is presented and analyzed below.

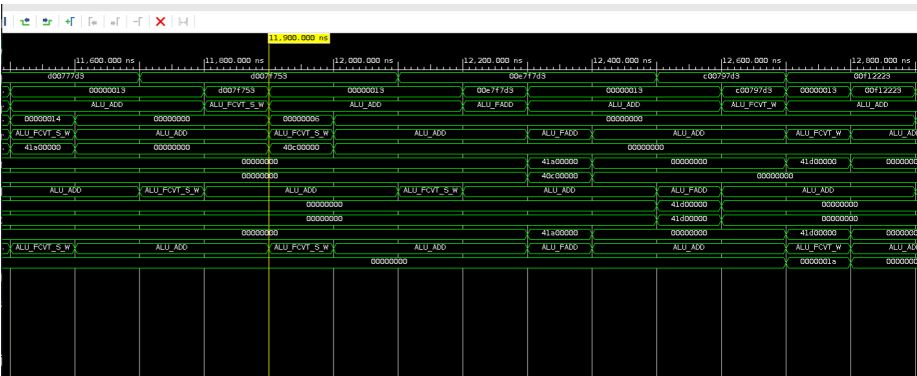


Figure 11.1 – Simulation Verification Results

Explanation

The simulation waveform shown in Figure 11.1 illustrates the execution of the test program developed in Section 8.8.

```
1 int test_fpu_logic_add() {
2     volatile int a = 20;
3     volatile int b = 6;
4 }
```

```

5     float f1 = (float)a; // fcvt.s.w
6     float f2 = (float)b; // fcvt.s.w
7
8     float sum = f1 + f2; // fadd.s
9
10    return (int)sum;      // fcvt.w.s
11 }
12
13 int main() {
14     volatile int result = test_fpu_logic_add();
15     return 0;
16 }

```

Listing 11.1 – Floating-point addition test in C

As observed in the waveform, when the ALU_FCVT_S_W operation is executed, the integer value 0x00000014 is converted into the floating-point representation 0x41A00000 (20.0). Similarly, the value 0x00000006 is converted into 0x40C00000 (6.0). These conversions occur in the first stage of the execution pipeline.

The converted operands propagate through the execution stages. When the ALU_FADD instruction enters the execution stage, it receives the required operands via forwarding from the memory stage and execution stage 3, ensuring correct data availability without pipeline stalls.

After the execution of the ALU_FADD operation, the resulting floating-point value 0x41D00000 (26.0) is forwarded to the ALU_FCVT_W unit. This unit converts the floating-point result back to an integer, producing the final result 0x0000001A (26).

The simulation results confirm the correct functionality of the floating-point unit, including proper operand conversion, execution, forwarding, and result generation within the pipeline.

11.1.2 Verification II

The following code snippet presents the implementation and evaluation of multiple floating-point operations.

```

1 void test_fpu_logic_ref()
2 {
3     volatile int a = 20;
4     volatile int b = 6;
5     float f1 = (float)a; // fcvt.s.w
6     float f2 = (float)b; // fcvt.s.w
7     float result = f1/f2; // fdiv.s    3.3333333
8     float sum = f1 + f2; // fadd.s    26
9     printf("The division result is : \n
10           0x%08X\n",*(uint32_t*)&result);
11     printf("The addition result is : 0x%08X\n",*(uint32_t*)&sum);

```

```

11     if (result < sum) { // 3.3333333 < 26
12
13         volatile float f4 = f2 * result; // fmul.s
14         printf("The multiplication result is : \n
15             0x%08X\n",*(uint32_t*)&f4);
16     } else {
17         volatile int d = (int)result; // fcvt.w.s
18         printf("The converted integer result is : \n
19             0x%08X\n",*(uint32_t*)&d);
20     }
21 }

```

Listing 11.2 – Floating-point multiple operation test in C

Reference Verification

The following results were obtained by executing the C-based reference implementation and serve as a baseline for validation:

- The division result is: 0x40555555
- The addition result is: 0x41D00000
- The multiplication result is : 0x41A00000

Functional Verification

The following waveform reveals the simulation results:

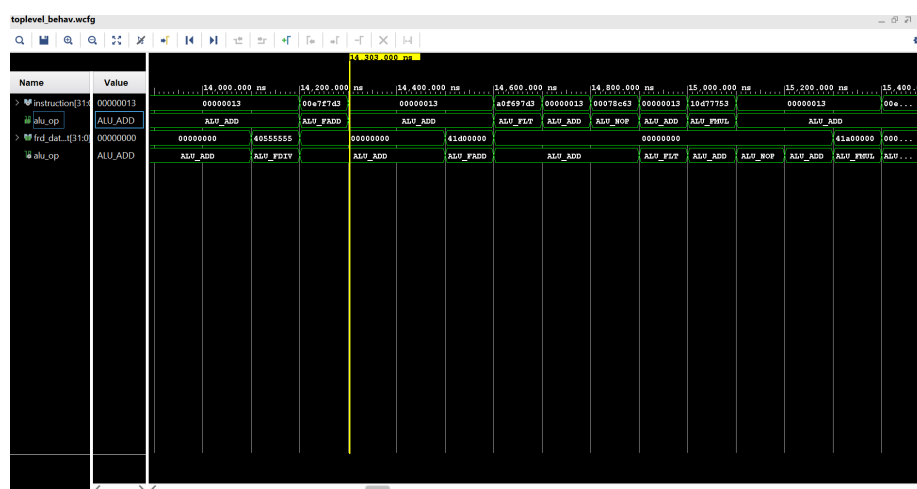


Figure 11.2 – A.Functional Verification Results

Signal	Value	Signal	Value	Signal	Value	Signal	Value	Signal	Value
mem_stg2	MEMOP_TYPE_NONE	mem_stg3	MEMOP_TYPE_NONE	alu_op_in	00000000	fcs1[31:0]	00000000	fcs2[31:0]	00000000
result	0								

Figure 11.3 – B.Functional Verification Results

The functional verification results are consistent with the reference output, indicating the correct operation of the floating-point unit. All the floating point instructions were correctly decoded and implemented.

11.2 FPGA Verification Results

The design was synthesized and implemented on the Xilinx Zynq-7000 platform (XC7Z020-1CLG400C) as well as the Ultra96-V2 development board, which is based on the AMD Xilinx Zynq UltraScale+ MPSoC architecture.

The design was successfully implemented to operate at a clock frequency of 125 MHz on the target FPGA platform. The main memory is 512 KB.

The results of the test application were transmitted and displayed via a UART interface. For integer outputs, the results were converted into string format using an `int_to_str` function and printed to the terminal.

All implemented arithmetic operations were successfully verified, and the observed results matched the expected values. However, when converting floating-point results to integers using the `fcvt.w.s` instruction, the outputs are truncated toward zero, which lead to a loss of fractional precision in the displayed results.

To accurately represent floating-point values, the results can be printed in hexadecimal format, as the FPU stores data in IEEE-754 representation. This can be achieved using a `hex_to_string` function, allowing the exact floating-point values to be observed. Additionally, these hexadecimal values can be converted to decimal form for verification purposes.

11.3 Observed Limitations

Despite the successful integration of the floating-point unit (FPU), several limitations have been identified in the current design.

- **Division Latency:**

The implemented division unit introduces significant latency, which requires more than 17 clock cycles to complete. This results in pipeline stalls during division operations, which can negatively impact overall performance.

- **Limited ISA Support:**

A reduced RISC-V floating-point instruction set has been implemented. As a result, the current design does not fully support all floating-point operations defined in the standard ISA.

- **Exclusion of FCSR**

While rounding modes and exception flags are supported within the FPU, a dedicated Floating-Point Control and Status Register (FCSR) has not been incorporated in the design.

- **Operating Frequency Constraint:**

The design achieves a maximum operating frequency of 125 MHz on the target FPGA platform. This limitation is primarily influenced by the critical paths within the execution stage and the integration of floating-point units.

- **Memory Utilization:**

Further optimization of memory resources is required. The current implementation does not fully optimize the usage of available memory blocks, which may impact scalability and efficiency.

Chapter 12

Conclusion and Future Work

12.1 Summary of Contributions

This thesis began with the analysis of an existing floating-point arithmetic unit in which all arithmetic operations were implemented using combinational logic. While the design had been functionally verified through simulation, it had not been validated on hardware.

As an initial contribution, the design was implemented on an FPGA platform, which revealed significant timing limitations, with the maximum achievable clock frequency reduced to approximately 5 MHz due to long combinatorial paths. This observation highlighted the need for architectural improvements.

To address these limitations, a pipelined floating-point unit architecture was proposed and developed, significantly improving timing performance and enabling efficient execution of floating-point operations.

Analysis of the design revealed that the critical path was dominated by the division operation. To mitigate this bottleneck, two alternative division architectures were designed: a Newton–Raphson (NR) based approximate division module and an iterative division module.

Considering factors such as accuracy, hardware complexity, and ease of integration into the pipeline, the iterative division approach was selected to replace the original combinational division logic.

The arithmetic units were redesigned into a three-stage pipelined architecture consisting of extraction, computation, and normalization stages. The complete system was validated through both simulation and hardware implementation.

The optimized design achieves an operating frequency of approximately 55 MHz, representing a significant improvement over the initial architecture.

In the next step, a subset of the RISC-V instruction set architecture (ISA) was integrated to enable proper operation of the floating-point unit. A test program was developed in C and compiled into instruction-level code using the RISC-V toolchain.

Subsequently, a coefficient file (.coe) was generated, containing the corresponding RISC-V instructions, which were used to initialize the instruction memory for execution on the processor.

In the final stage, the designed floating point instruction set architecture was integrated into the NOVACore in-order RISC-V processor. A dedicated floating-point register file was introduced to support floating-point operations.

All hazards arising from the integration were addressed using a combination of forwarding mechanisms, insertion of NOPs where necessary, and appropriate pipeline stalling logic to ensure correct execution.

The final NOVACore in-order processor was verified through both simulation and FPGA-based implementation. The operating frequency was 125MHz. The results demonstrate correct decoding of floating-point instructions, proper execution of operations, and accurate final outputs consistent with the expected behavior.

Thus, the presented work successfully realizes a RISC-V compatible IEEE-754 floating-point unit on FPGA, validating its functionality, performance, and integration within a complete processor architecture.

12.2 Future Improvements

Several enhancements can be made to further improve the performance and functionality of the proposed design:

- **Optimized Newton–Raphson Division:**

A more efficient implementation of the Newton–Raphson division algorithm can be explored by utilizing fused multiply-add (FMA) operations. This approach can reduce latency and improve numerical accuracy.

- **Extension of Instruction Set:**

The processor can be extended to support a broader subset of the RISC-V ISA, thereby improving its computational capabilities and flexibility.

- **Advanced Instruction Support:**

Support for additional instruction extensions, such as atomic operations and other advanced RISC-V extensions, can be incorporated to enhance functionality and enable more complex applications.

- **Support for Multiple Precisions:**

Currently, only single-precision floating-point operations are implemented.

The design can be extended to support half-precision (16-bit) and double-precision (64-bit) formats, improving versatility across different application domains.

- **Extension to Ternary Number Systems:**

Currently, the processor operates using conventional binary arithmetic. As a potential improvement, alternative number representations such as ternary or Canonical Signed Digit (CSD) encoding can be explored. These representations can reduce the number of non-zero digits in arithmetic operations, potentially improving computational efficiency and reducing hardware complexity in certain operations. [33] [34][35]

- **Support for Vector Operations (RVV):**

The current design supports only scalar floating-point operations, where one operation is performed per instruction. As a potential enhancement, support for vector operations using the RISC-V Vector Extension (RVV) can be incorporated. Vector instructions enable the simultaneous processing of multiple data elements, significantly improving computational throughput. This is particularly beneficial for applications such as scientific computing, signal processing, and machine learning, where large datasets are processed in parallel. [32] [36]

List of Figures

1.1 Stages of Development	4
2.1 Singe Cycle Processing	8
2.2 Pipelined Processing	9
2.3 Pipeline Stages	9
2.4 IEEE 754 Floating-Point 32 Bit Single Precision Layout	13
2.5 FPGA Architecture	15
3.1 Initial Architecture	17
3.2 Zed Board	19
3.3 Test Setup.	21
3.4 Correct arithmetic operation waveform demonstrating FPU functionality.	22
3.5 Timing analysis showing failure at 100 MHz.	22
3.6 Timing analysis showing successful timing closure at 5 MHz.	23
4.1 Execution Stage	29
4.2 Three Staged Execution Stage	30
5.1 NR Division Architecture	35
5.2 Iterative Restoring Division Algorithm	39
5.3 NR Division Architecture Memory Utilisation	45
5.4 Restoring Iterative Division Architecture Memory Utilisation	45
5.5 Final Division Architecture	47
6.1 Implementation Results using ILA	51
6.2 Final FP Arithmetic Unit Architecture	52
9.1 NovaCore Architecture	67
10.1 Execution Architecture	72
10.2 Architecture of the NovaCore processor with integrated floating-point unit (FPU).	76

10.3 Final NovaCore architecture.	76
11.1 Simulation Verification Results	77
11.2 A.Functional Verification Results	79
11.3 B.Functional Verification Results	80
A.1 ILA Debugging of Special Division Operations.	88
A.2 ILA Debugging of Negative Numbers.	88
A.3 ILA Debugging of Results.	89

List of Tables

3.1	ZedBoard (Zynq-7000 Development Board) Specifications	20
5.1	Verification results for exact division cases	37
5.2	Verification results for IEEE-754 special cases	38
5.3	Verification results for high-precision floating-point division	38
5.4	FPGA verification results for floating-point operations	42
5.5	Verification of basic floating-point operations	43
5.6	Verification of IEEE-754 special cases	43
5.7	Additional floating-point verification tests	43
5.8	ULP error analysis of the implemented floating-point unit	44
5.9	Verification of basic floating-point operations	44
5.10	Verification of IEEE-754 special cases	44
5.11	Additional floating-point verification tests	44
5.12	ULP error analysis of the implemented floating-point unit	45
5.13	Comparison between Newton–Raphson and Iterative Division Architectures	46
7.1	Single-precision floating-point instructions in the RISC-V F-extension	54
7.2	R-Type floating-point instruction format in the RISC-V F-extension . .	55
7.3	Floating-point instructions from the RISC-V F-extension not implemented in the proposed architecture	56
8.1	Standard RISC-V Extensions	58
8.2	Important compiler and linker flags	59

Appendix A

Extra Data Supporting the Architecture

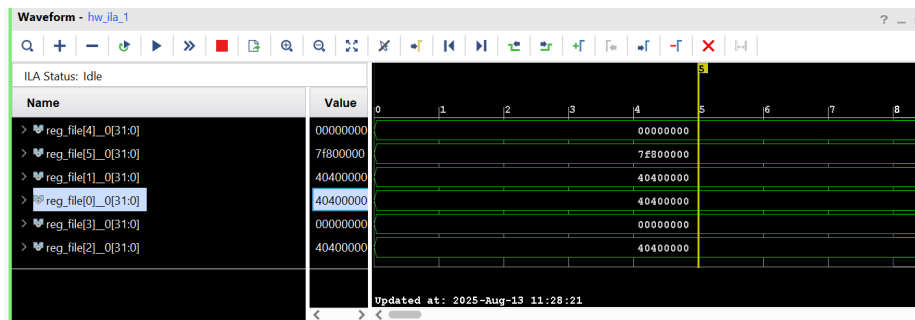


Figure A.1 – ILA Debugging of Special Division Operations.

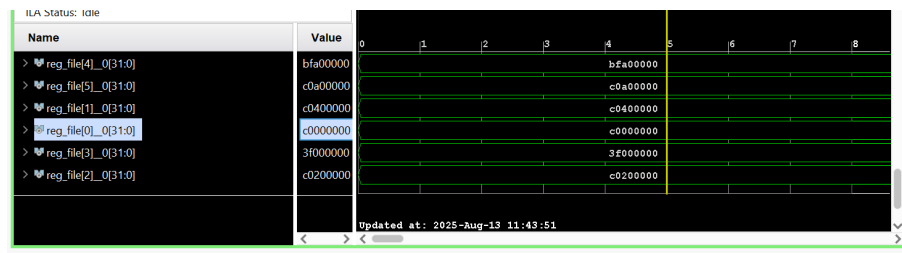


Figure A.2 – ILA Debugging of Negative Numbers.

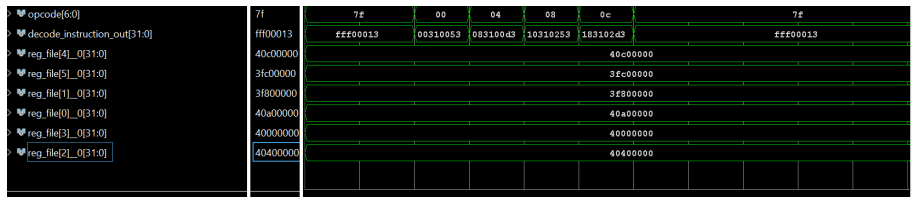


Figure A.3 – ILA Debugging of Results.

```

1 float test_fpu_logic_div() {
2     volatile float a = 20.5f;
3     volatile float b = 6.5f;
4     float result = a/b;
5     //printf("The division result is : %f",result);
6     return result;
7 }
8
9 int main() {
10
11     volatile float result = test_fpu_logic_div();
12     return 0;
13 }

```

Listing A.1 – Floating-point division test in C

```

1
2 test_fpu_logic_div:
3 addi    sp,sp,-32
4 sw      ra,28(sp)
5 sw      s0,24(sp)
6 addi    s0,sp,32
7
8 ' ' '
9 lui     a5,%hi(.LC2)
10 flw     fa5,%lo(.LC2)(a5)
11 fsw     fa5,-24(s0)
12
13 lui     a5,%hi(.LC3)
14 flw     fa5,%lo(.LC3)(a5)
15 fsw     fa5,-28(s0)
16
17 flw     fa4,-24(s0)
18 flw     fa5,-28(s0)
19 fdiv.s  fa5,fa4,fa5
20 fsw     fa5,-20(s0)
21
22 flw     fa5,-20(s0)
23 fmv.x.s a0,fa5
24

```

```

25  lw      ra,28(sp)
26  lw      s0,24(sp)
27  addi    sp,sp,32
28  jr      ra
29
30  .size    test_fpu_logic_div, .-test_fpu_logic_div
31  .align   2
32
33  .globl   main
34  .type    main, @function
35  ' ' '
36
37  main:
38  addi     sp,sp,-32
39  sw       ra,28(sp)
40  sw       s0,24(sp)
41  addi     s0,sp,32
42
43  ' ' '
44  call     test_fpu_logic_div
45  fmv.s.x  fa5,a0
46  fsw      fa5,-20(s0)
47
48  li       a5,0
49  mv       a0,a5
50
51  lw       ra,28(sp)
52  lw       s0,24(sp)
53  addi     sp,sp,32
54  jr       ra
55  .size    main, .-main
56  .section .rodata
57  .align   2
58  ' ' '
59  .LC2:
60  .word    1101266944
61
62  ' ' '
63  .align   2
64  ' ' '
65  .LC3:
66  .word    1087373312
67
68  ' ' '
69  .ident   "GCC: (14.2.0+19) 14.2.0"
70  .section .note.GNU-stack,"",@progbits
71  ' ' '

```

Listing A.2 – RISC-V assembly generated for floating-point division

Acronyms

FPGA Field Programmable Gate Array. 2, 3

FPU Floating Point Unit. 6

IEEE 754 Institute of Electrical and Electronics Engineers Standard 754. 4

IP Intellectual Property. 22

RAW Read After Write. 5

RISC-V Reduced Instruction Set Computer – Five. 2

RTL Register Transfer Level. 2

ULP Unit in the Last Place. 38

VHDL Very High-Speed Integrated Circuit Hardware Description Language. 3

Bibliography

- [1] K. Wali, "Enhancing performance with floating-point instruction integration in risc-v architectures," *Journal of Advances in Developmental Research (IJAIDR)*, vol. 14, no. 2, pp. 1–13, 2023. [Online]. Available: www.ijaidr.com (cit. on pp. 1, 54).
- [2] A. Varkey, "A systemc-based implementation of an ieee 754 pipelined floating-point unit with zynq fpga verification," in *Master's Thesis, University of Erlangen, Dept. of Computer Science, June 2025, University of Erlangen, Dept. of Computer Science, June 2025.*, 2025 (cit. on pp. 6, 17).
- [3] L. Poli, S. Saha, X. Zhai, and K. D. McDonald-Maier, "Design and implementation of a risc v processor on fpga," in *2021 17th International Conference on Mobility, Sensing and Networking (MSN)*, 2021, pp. 161–166 (cit. on pp. 6, 25).
- [4] F. Hussain and S. Sarkar, "Design and fpga implementation of five stage pipelined risc -v processor," in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, 2024, pp. 1–6 (cit. on pp. 6, 10, 11).
- [5] M. Tambaro, S. Ceopi, A. Gallacci, A. La Gala, M. Malanchini, and M. De Matteis, "500 mhz cmos 28-nm floating point arithmetic unit for 32-bit risc-v microprocessors," in *2024 19th Conference on Ph.D Research in Microelectronics and Electronics (PRIME)*, 2024, pp. 1–4 (cit. on p. 6).
- [6] X. Wu, Y. Wang, T. Zhao, J. Qin, Z. Liang, and J. Fu, "Pvu: Design and implementation of a posit vector arithmetic unit (pvu) for enhanced floating-point computing in edge and ai applications," 2022 (cit. on p. 6).
- [7] M. Kovač et al., "Faust: Design and implementation of a pipelined risc-v vector floating-point unit," *Microprocessors and Microsystems*, vol. 97, p. 104762, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0141193312300008X> (cit. on p. 6).
- [8] G. G. P, J. S, and K. R. S, "An area optimized floating-point coprocessor for risc-v processor," in *2023 International Conference on Control, Communication and Computing (ICCC)*, 2023, pp. 1–5 (cit. on pp. 7, 54).

- [9] D. Zoni, A. Galimberti, and W. Fornaciari, "An fpu design template to optimize the accuracy-efficiency-area trade-off," DEIB, Politecnico di Milano, Italy (cit. on p. 7).
- [10] F. Ebrahimi-azandaryani and D. Fey, "Csd-driven speedup in risc-v processor," in *Design and Architecture for Signal and Image Processing*, J. Lorandel and A. Kamaleldin, Eds., Cham: Springer Nature Switzerland, 2025, pp. 3–15 (cit. on pp. 7, 67, 68).
- [11] N. Louvet, J.-M. Muller, and A. Panhaleux, "Newton-raphson algorithms for floating-point division using an fma," in *ASAP 2010 - 21st IEEE International Conference on Application-specific Systems, Architectures and Processors*, 2010, pp. 200–207 (cit. on pp. 7, 32).
- [12] S. Li. "Risc-v based cpu design with logisim [part 7]." Updated April 6, 2018. Accessed: March 14, 2026. [Online]. Available: <https://lishixuan001.com/posts/3920/> (cit. on pp. 8–10).
- [13] Z. Qin, "Design and hazard solving of five-stage pipeline risc-v processor structure," *Applied and Computational Engineering*, vol. 36, pp. 198–203, Feb. 2024 (cit. on pp. 10, 11).
- [14] openhwgroup, "Https://docs.openhwgroup.org/projects/cv32e40p-user-manual/en/cv32e40p_v1.8.3/index.html," [online], 2023 (cit. on pp. 11, 12).
- [15] IEEE 754 Committee, "IEEE standard for floating-point arithmetic," Micro-processor Standards Committee of the IEEE Computer Society, New York, NY, IEEE Std 754-2008, Aug. 2008, (committee member and contributor), pp. 1–70 (cit. on pp. 12, 25).
- [16] M. Cowlishaw, "Decimal floating-point: Algorithm for computers," in *Proceedings 2003 16th IEEE Symposium on Computer Arithmetic*, 2003, pp. 104–111 (cit. on p. 12).
- [17] S. Hollasch, *Ieee standard 754 floating point numbers*, <http://steve.hollasch.net/cgiindex/coding/ieeefloat.html>, Accessed: 2026-03-28, 1998 (cit. on pp. 13, 25).
- [18] A. Boutros and V. Betz, "Fpga architecture: Principles and progression," *IEEE Circuits and Systems Magazine*, vol. 21, no. 2, pp. 4–29, 2021 (cit. on pp. 15, 16).
- [19] M. D. Inc, "https://www.mirabilisdesign.com/working-applications-of-fpga/" (cit. on p. 15).
- [20] zedboard reference-manual, "https://digilent.com/reference/programmable-logic/zedboard/reference-manual" (cit. on pp. 19, 20).

- [21] R. Trüby, M. Fleury, and A. Biere, “Verifying floating-point commutativity with grs,” in <https://cca.informatik.uni-freiburg.de/papers/TrubyFleuryBiere-SAT-Competition-2023-benchmarks.pdf>, Freiburg, Germany, 2023 (cit. on p. 28).
- [22] NAJIB GHATTE, SHILPA PATIL, DEEPAK BHOIR, “Single precision floating point division,” Fr. Conceicao Rodrigues College of Engineering, Fr. Agnel Ashram, Bandstand, Bandra (W), Mumbai: 400 050, India, Tech. Rep., 2014, [Online]. Proceedings of Fifth IRF International Conference, 10th August 2014, Goa, India, ISBN: 978-93-84209-45-2 (cit. on pp. 31, 32, 40).
- [23] S. Halder, A. Hasnat, A. Hoque, D. Bhattacharjee, and M. Nasipuri, “Pipelining based floating point division: Architecture and modeling,” *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, vol. 3, no. 1, pp. 15–19, Jun. 2013. [Online]. Available: www.ijitee.org (cit. on pp. 32–35).
- [24] A. Parker and J. Hamblen, “Optimal value for the newton-raphson division algorithm,” *Information Processing Letters*, vol. 42, no. 3, pp. 141–144, 1992. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/002001909290137K> (cit. on p. 33).
- [25] J. D. Bruguera, “Radix-64 floating-point division and square root: Iterative and pipelined units,” *IEEE Transactions on Computers*, vol. 72, no. 10, pp. 2990–3001, 2023 (cit. on p. 33).
- [26] S. Abdel-Hafeez, “A fast and cost-effective integer restoring division hardware,” *Computers and Electrical Engineering*, vol. 116, p. 109 221, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0045790624001496> (cit. on p. 39).
- [27] XRAwati, “Intellectual property core (ip core),” in <https://www.techtarget.com/whatis/definition/IP-core-intellectual-property-core>, 2022 (cit. on p. 48).
- [28] AMD, “Vivado design suite tutorial: Designing ip subsystems using ip integrator (ug995),” Advanced Micro Devices (AMD), Tech. Rep., Apr. 2022, [Online]. Available: https://www.xilinx.com/support/documents/sw_manuals/xilinx2022_1/ug995-vivado-ip-subsystems-tutorial.pdf. [Accessed: Mar. 29, 2026] (cit. on pp. 48, 49).
- [29] Y. Gao, W. Qian, and E. Cui, “Risc-v isa extension toolchain supports: A survey,” in *Proceedings of the 2023 4th International Conference on Computing, Networks and Internet of Things*, ser. CNIOT ’23, Xiamen, China: Association for Computing Machinery, 2023, pp. 924–929. [Online]. Available: <https://doi.org/10.1145/3603781.3603942> (cit. on pp. 57, 58).

- [30] K. Shaikh et al., “Analyzing risc-v compiler toolchain by adopting topic modeling,” *IEEE Access*, vol. 14, pp. 33 178–33 198, 2026 (cit. on p. 57).
- [31] Fraunhofer IPMS, “Risc-v processor core for functional safety,” Fraunhofer Institute for Photonic Microsystems IPMS, Dresden, Germany, Whitepaper, 2022. [Online]. Available: www.ipms.fraunhofer.de (cit. on p. 58).
- [32] A. Fernandes, L. Crespo, N. Neves, P. Tomás, N. Roma, and G. Falcao, “Functional validation of the risc-v unlimited vector extension,” *IEEE Embedded Systems Letters*, vol. 17, no. 1, pp. 2–5, 2025 (cit. on pp. 59, 84).
- [33] Y. He et al., “A fast and energy efficient binary-to-pseudo csd converter,” in *2015 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2015, pp. 838–841 (cit. on p. 84).
- [34] Y. Tanaka, “Efficient signed-digit-to-canonical-signed-digit recoding circuits,” *Microelectronics Journal*, vol. 57, pp. 21–25, Nov. 2016 (cit. on p. 84).
- [35] L. Turner, P. Graumann, and S. Gibb, “Bit-serial fir filters with csd coefficients for fpgas,” vol. 975, pp. 311–320, Jan. 2006 (cit. on p. 84).
- [36] *Journal of advances in developmental research (ijaidr)*, volume 14, issue 2, <http://www.ijaidr.com>, E-ISSN: 0976-4844, Jul. 2023 (cit. on p. 84).